Meydan AI Handover

Website Generator — full handover bundle

Package Handover_Meydan_2026-04

Delivered April 2026 · reviewed 2026-04-27

Audience A developer who has not seen the product before

Live https://meydanfzai.com (AWS ap-southeast-1, fronted by CloudFront)

Vendor lead Ashutosh — ashuken266@gmail.com

1. System Overview

What it is

An AI-powered website generator that produces a deployed, brand-consistent business website from a short brief in under 90 seconds.

Who uses it

Small and mid-sized businesses licensed in Meydan Free Zone (UAE), who sign up with their company details and trade-licence number and need a live website in hours rather than weeks.

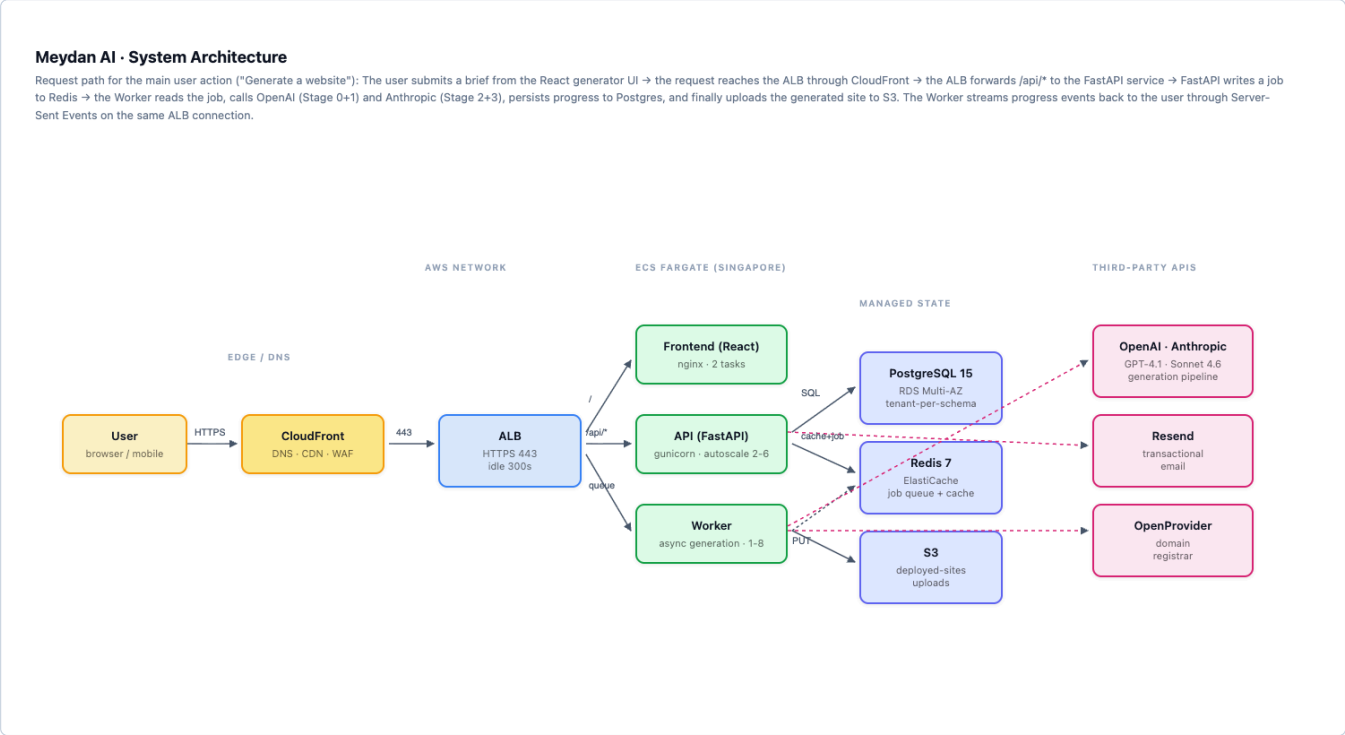
Tech stack

Layer	Choice
Frontend	React 18 (Create React App), React Router v6, GSAP animations, Monaco editor, custom CSS
Backend	Python 3.12, FastAPI (async), gunicorn + UvicornWorker
Database	PostgreSQL 15 (RDS Multi-AZ), tenant-per-schema isolation
Cache + queue	Redis 7 (ElastiCache)
Hosting	AWS ECS Fargate (`ap-southeast-1`), ALB, S3 static hosting for deployed customer sites
Edge	AWS CloudFront (CDN + SSL via ACM) + AWS WAF
DNS	AWS Route 53 (apex `meydanfzai.com` and customer custom domains)
Email	Resend (transactional)
AI models	OpenAI `gpt-4.1-mini` (enhance) + `gpt-4.1` (plan) and Anthropic `claude-sonnet-4-6` (HTML + JS generation)

Top three external dependencies

Dependency	What breaks if it goes down	Mitigation in code
Anthropic API	Stage 2 (HTML) and Stage 3 (JS) generation halts. Users see an error page before any HTML is produced.	Circuit breaker opens after 5 failures (60s recovery); user gets a clean retry message.
OpenAI API	Stage 0 (enhance) and Stage 1 (plan) halt; pre-generation moderation fails closed → all new prompts blocked.	Same circuit-breaker pattern. Operator can flip `MODERATION_LLM_ENABLED=false` for keyword-only mode in an outage.
AWS CloudFront	The CDN edge for the platform and for every customer-deployed custom domain. If CloudFront has a region-wide degradation, customer sites slow / 5xx.	CloudFront has its own multi-region failover; outages are rare. Origin (ALB / S3) remains directly reachable, so an emergency runbook can re-point DNS straight at origin while CloudFront recovers.

2. Architecture Diagram



Editable source: `assets/diagrams/architecture.html` — open in any browser to inspect or edit.

Caption — request path for the main user action

A user submits a generation brief from the React UI. The request leaves their browser as HTTPS to CloudFront (the authoritative DNS), which proxies it to the ALB in `ap-southeast-1`. The ALB routes by path:

- `/` and the dashboard → **Frontend** service (nginx + React build, 2 tasks)
- `/api/*` → **API** service (FastAPI + gunicorn, 2-6 tasks autoscaling on CPU)

The API persists the request to PostgreSQL, enqueues a generation job in Redis, and returns immediately. The user's browser opens a Server-Sent Events stream to the same ALB; the API streams progress events back as the job advances.

The **Worker** service consumes the Redis queue and runs the four-stage generation pipeline:

1. **Stage 0 (Enhance)** — GPT-4.1-mini expands the brief into a creative brief (~2 s, ~1.5K tokens)
2. **Stage 1 (Plan)** — GPT-4.1 produces a JSON plan with brand tokens and section layout (~7 s)
3. **Stage 2 (HTML)** — Claude Sonnet 4.6 emits the full Tailwind-styled HTML (~50 s, the heaviest stage)
4. **Stage 3 (JS)** — Claude Sonnet 4.6 emits vanilla JS interactions (~20 s)

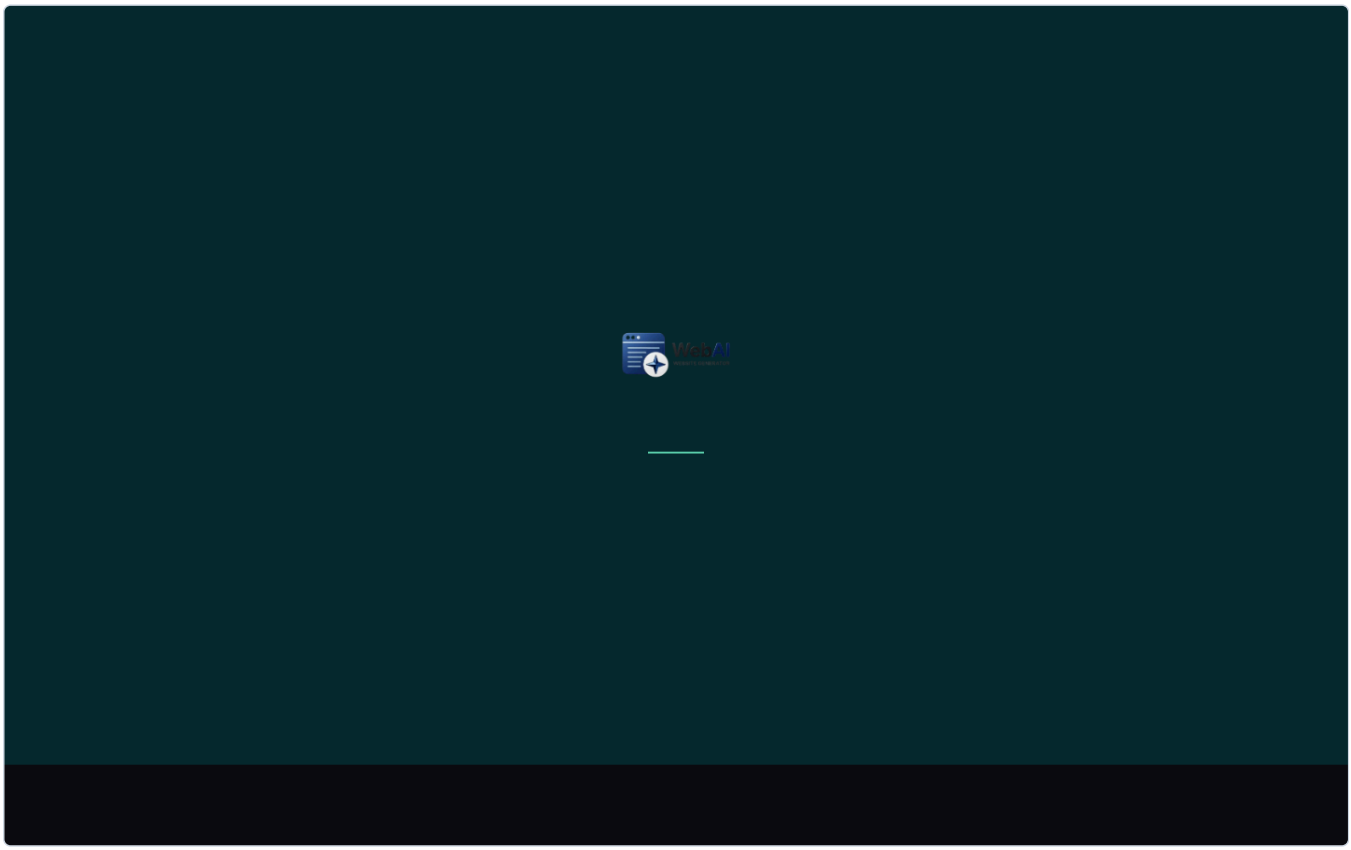
The Worker writes generated artefacts to a tenant schema in PostgreSQL and uploads the deployable artefact bundle to S3 (`meydan-prod-sites-sg`). The frontend then loads the result; the user can edit and redeploy from the editor.

3. User Workflow

End-to-end journey of a real user from first visit to a deployed website.

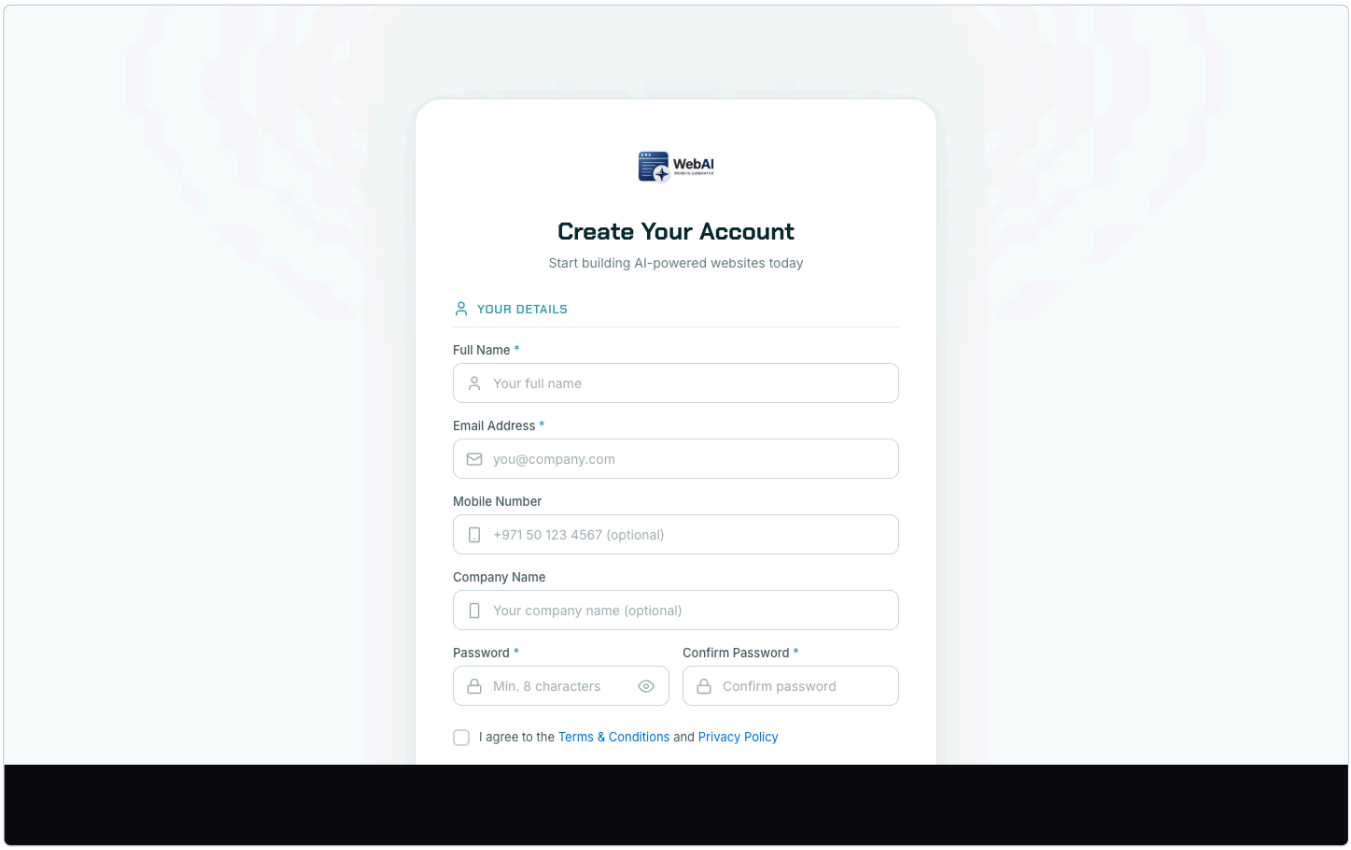
Step 1 — Land on the marketing site

User does: visits `https://meydanfzai.com`. **System does**: CloudFront serves the React-built landing page from the Singapore ALB. **User sees**: hero + animated demo + CTA to "Get Started".



Step 2 — Sign up

User does: clicks **Get Started** → fills the signup form (full name, work email, password, company name, trade-licence number). **System does**: `POST /api/auth/signup` validates the payload (12-char strong password, RFC-5321 email), creates a `public.users` row with `is_approved=false` and `tokens_balance=0`, creates a `tenant_<sanitised_email>` schema, and emails a verification link via Resend. **User sees**: a "check your inbox" screen.



Step 3 — Verify email

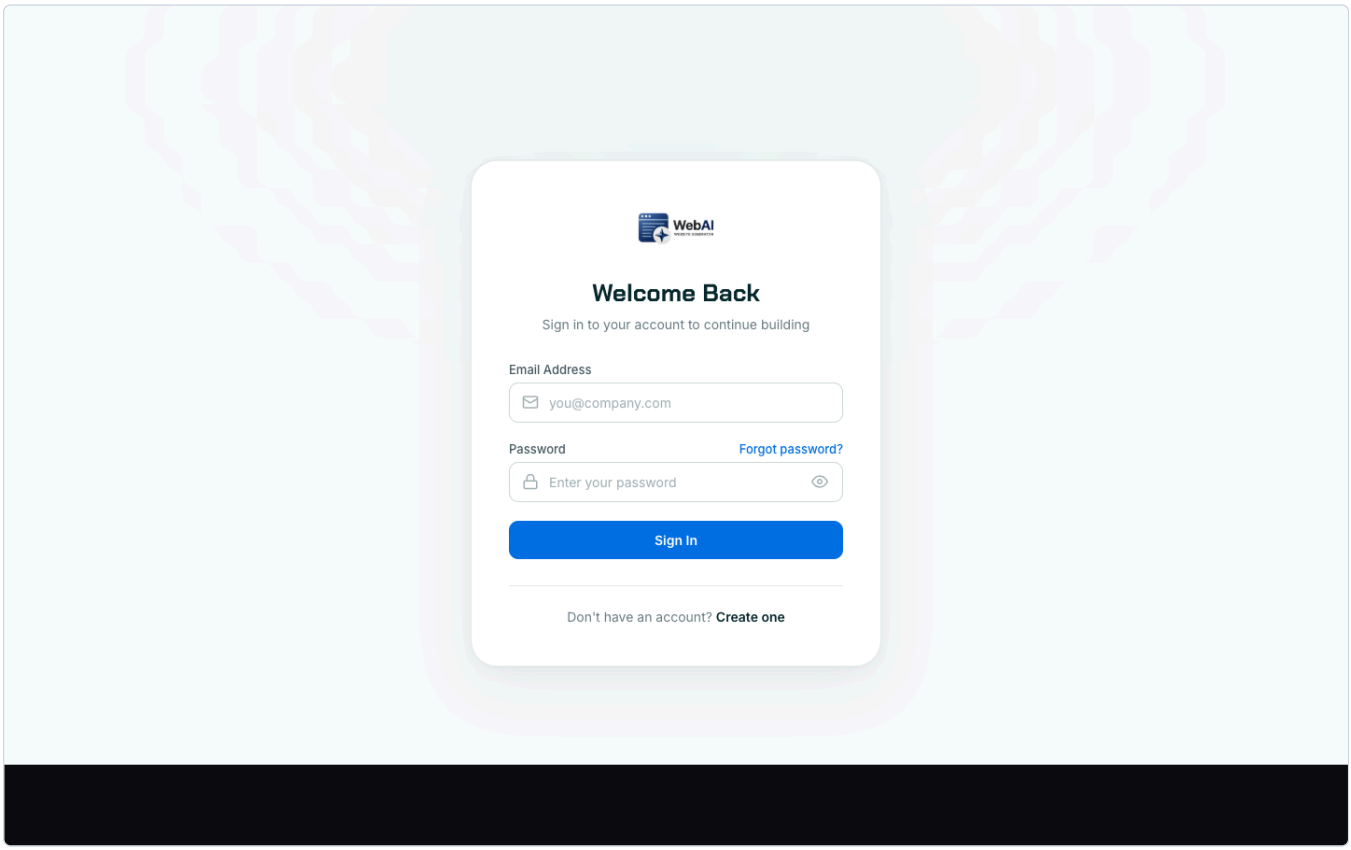
User does: clicks the link in their inbox. **System does**: `GET /api/auth/verify-email?token=…` flips `email_verified=true` and redirects to login. **User sees**: a success banner that says they can now log in once approved.

Step 4 — Pending approval

User does: tries to log in. **System does**: `POST /api/auth/login` returns a JWT, but the `pending_approval` flag in the response routes the UI to a "waiting room" page. No tokens to spend yet. **User sees**: a friendly screen explaining that an admin will approve them shortly.

Step 5 — Log in (post-approval)

User does: refreshes after the admin approves them. Logs in normally. **System does**: same `POST /api/auth/login`; this time `pending_approval` is false, JWT routes to `/dashboard`. **User sees**: the dashboard with an empty sites list and a primary CTA "Generate a new site".



Step 6 — Submit a brief

User does: clicks **Generate** → picks a niche from the dropdown → writes a 1-3 sentence brief about their business → clicks **Generate**. **System does**: `POST /api/generator/generate-stream` runs keyword + LLM moderation; if the prompt is safe, opens a Server-Sent Events stream and enqueues the four-stage pipeline. **User sees**: a streaming progress UI ("Enhancing brief…", "Planning sections…", "Designing pages…", "Adding interactions…").

Step 7 — Edit in the workspace

User does: lands in the editor; can chat with the AI to adjust copy, swap images, change colours; or open the Monaco code panel directly. **System does**: each chat instruction hits `POST /api/sections/{id}/edit`, which runs an LLM-based diff against the section and returns the patched HTML/CSS/JS. **User sees**: a live preview that updates as edits land.

Step 8 — Deploy

User does: clicks **Deploy**. **System does**: `POST /api/deploy` packages the site, uploads to `meydan-prod-sites-sg` S3 bucket with a unique slug, and returns the public URL `https://<slug>.s3-website.ap-southeast-1.amazonaws.com`. If the user has a custom domain, CloudFront DNS is updated and an ACM certificate is requested. **User sees**: a success modal with the live URL and a "View site" button.

Error states

Trigger	User-visible message	Recovery path
Pre-gen moderation rejects prompt	"[RESTRICTED] Content safety check is temporarily unavailable. Please try again shortly." (or category-specific reason if keyword check fired)	User edits the brief and retries. Operator can flip `MODERATION_LLM_ENABLED=false` for outages.
LLM circuit breaker open	"Generation service is recovering. Try again in 60 seconds."	60-second wait; recovery is automatic.
Token balance exhausted	"You've used your monthly generation quota. Contact support to top up."	Admin grants tokens via the admin panel.

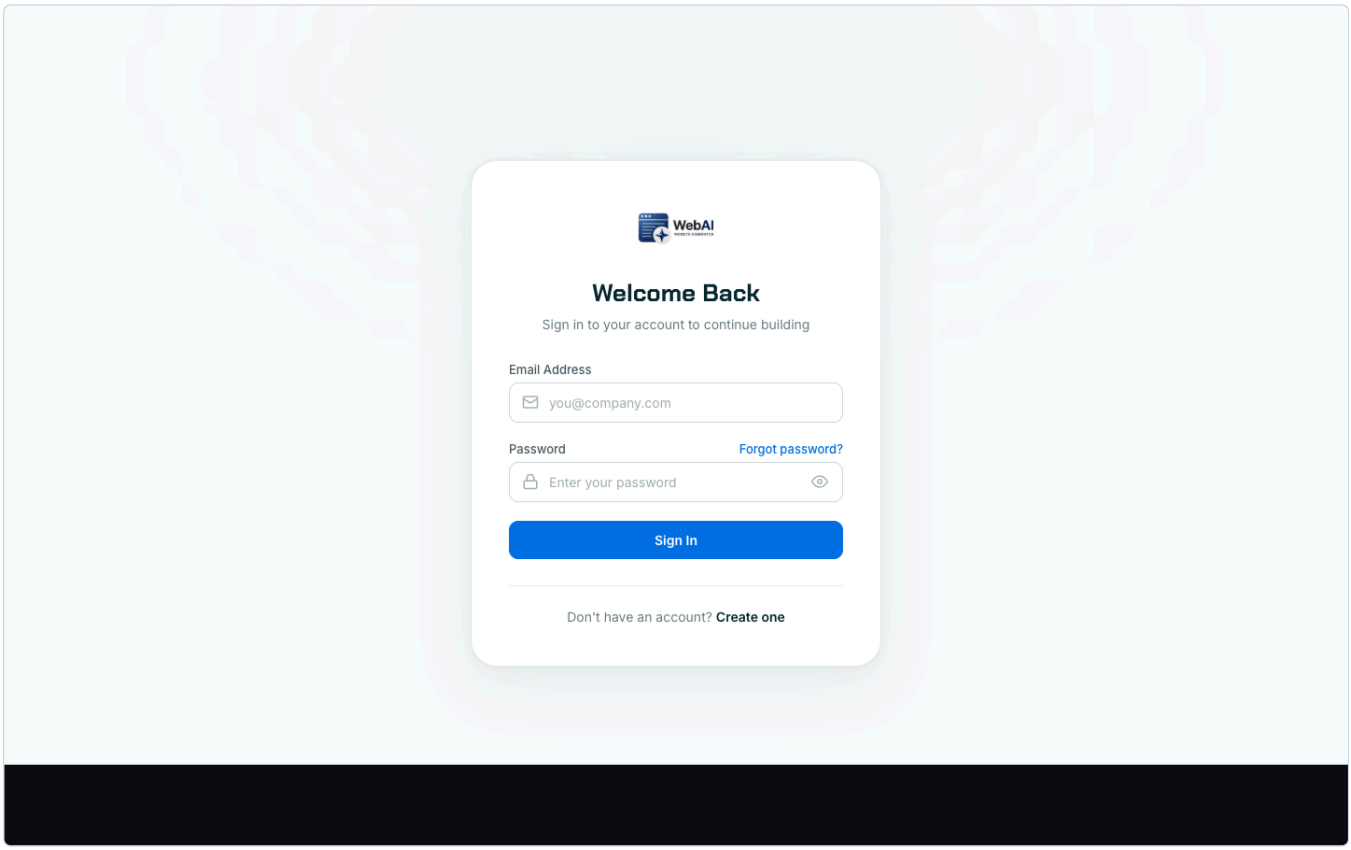
Trigger	User-visible message	Recovery path
Generation hits stage timeout (300s)	"Generation took too long. We've kept your draft — try editing or regenerating a single section."	Saved draft in DB; user can resume from the editor.
S3 deploy failure	"Deployment failed. Your site is saved — try again or contact support."	The plain message is shown; the full traceback is in CloudWatch with the correlation ID returned in the response header.

4. Admin Workflow

How an operator runs the product day-to-day. The admin panel is laid out in seven sections — **Dashboard**, **Sites & Generation**, **Domains**, **Users**, **Tokens**, **Operations**, **Analytics**.

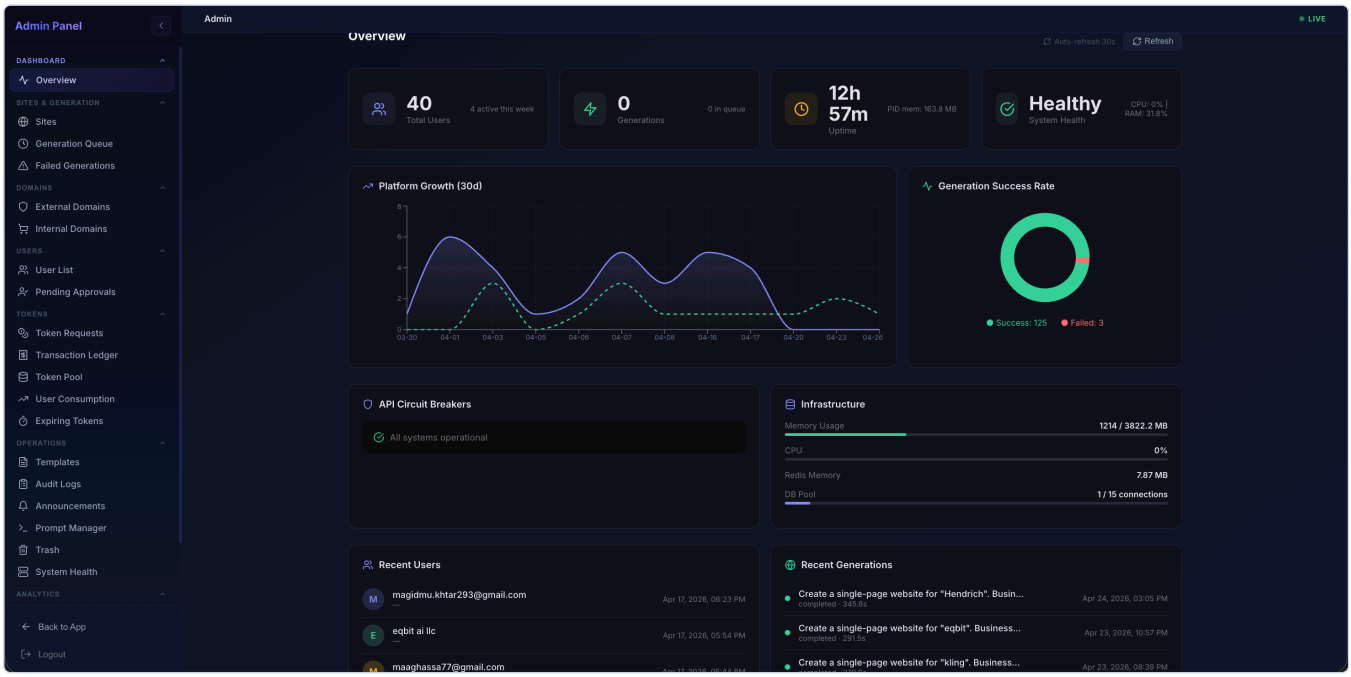
Step 1 — Log in as admin

Admin does: visits `https://meydanfzai.com/admin/login`, enters admin credentials. **System does**: `POST /api/auth/login` issues a JWT; if `is_admin=true` the UI exposes the admin sidebar at `/admin/*`.



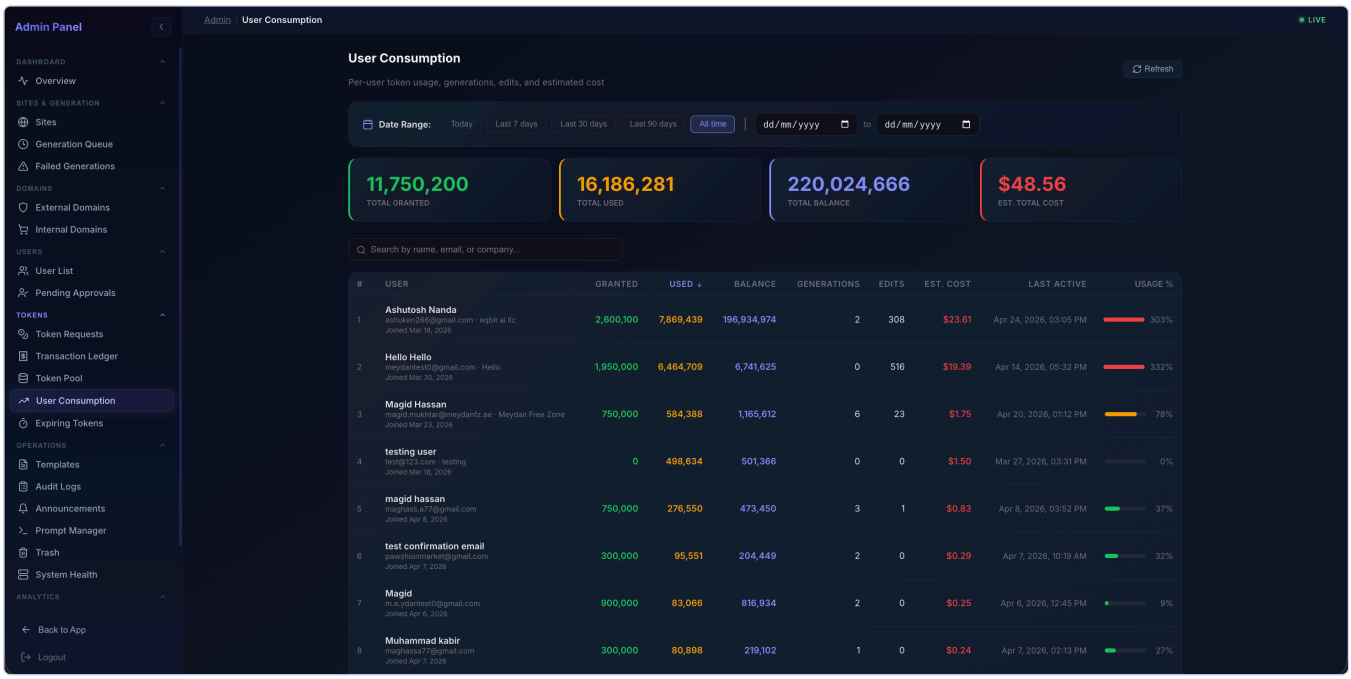
Step 2 — Read the Overview

Where: `/admin/overview` (default landing page). **What it shows**: total users, generations in queue, system uptime, system health, platform growth (30d), generation success rate, API circuit breakers, infrastructure usage (memory / CPU / Redis / DB pool), recent users, recent generations.



Step 3 — Find a user (User List & Consumption)

Where: `/admin/users` (sidebar **Users → User List**) — also doubles as the consumption table. **Admin does**: searches by email, name, or company; columns include granted tokens, used tokens, balance, generations, edits, last active, recent burn rate. **API**: `GET /api/admin/users?q=…&page=…` .

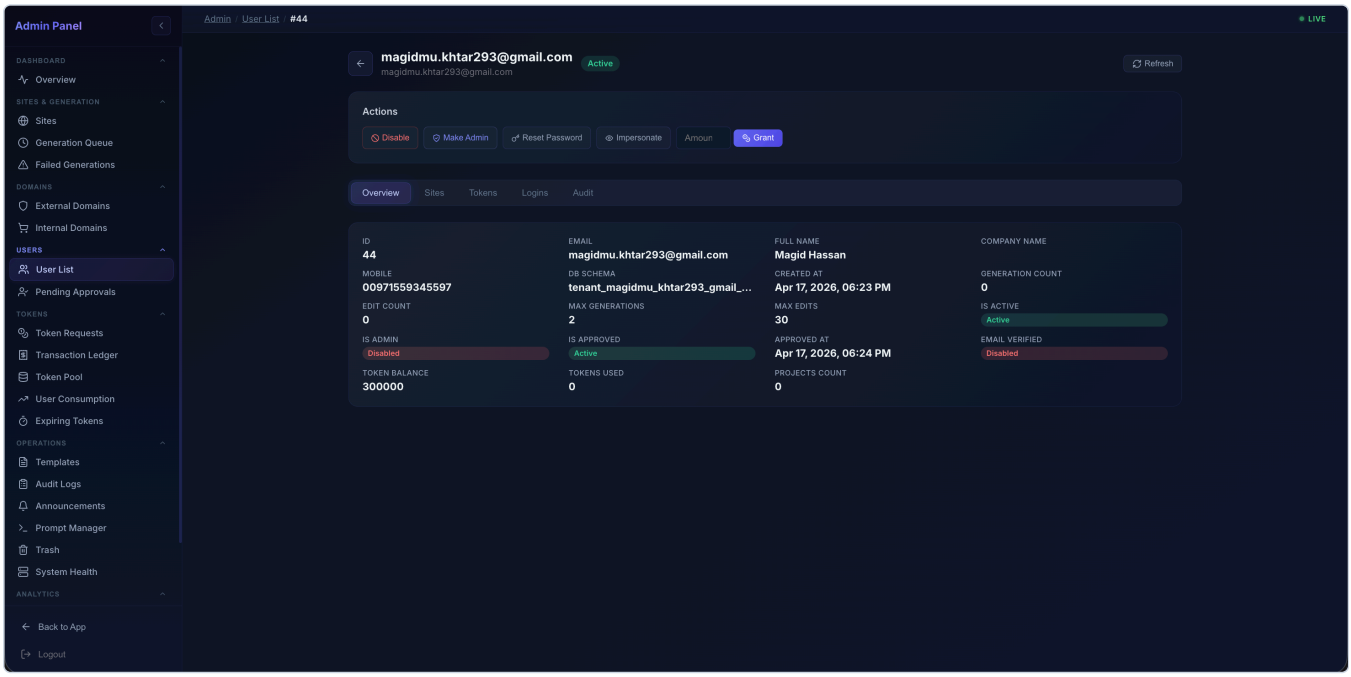


Step 4 — Open a user and adjust them

Where: click any user → `/admin/users/{id}` . **What it shows**: header with the user's email and a row of action buttons — **Disable**, **Make Admin**, **Reset Password**, **Impersonate**, **Annul**, **Grant**. Tabs underneath (Overview / Sites / Tokens / Logins / Audit). Profile data: full name, ID, mobile, schema name, generation count, edits, approval state, token balance, projects. **API**:

- `GET /api/admin/users/{id}` — detail
- `POST /api/admin/users/{id}/grant-tokens` — `{amount, reason}` for top-ups
- `POST /api/admin/users/{id}/disable` — suspend user

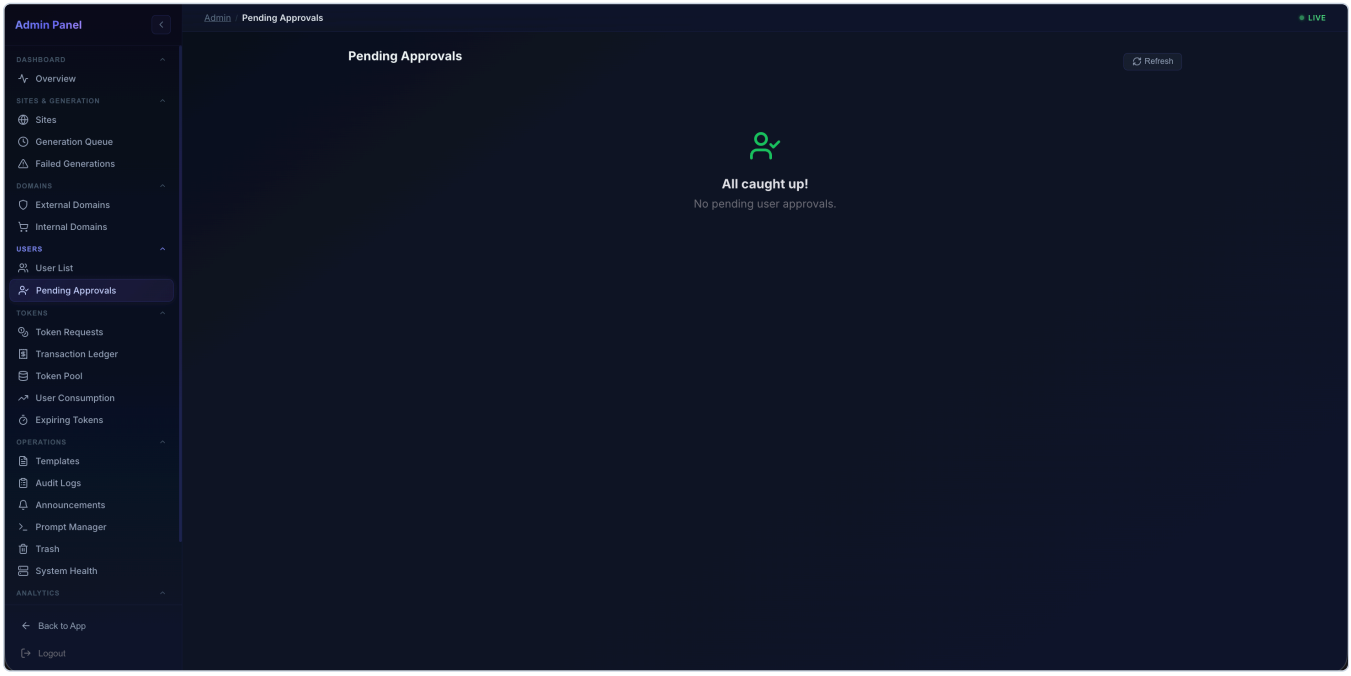
- `POST /api/admin/users/{id}/make-admin` — flip `is_admin=true`
- `POST /api/admin/users/{id}/impersonate` — issue an impersonation JWT (audit-logged)
- `POST /api/admin/users/{id}/annul` — wipe user's tenant schema (irreversible — confirmation modal)



Step 5 — Approve new signups

Where: `/admin/users/pending-approvals`. **What it shows**: queue of users with `is_approved=false`. Empty when caught up — the screenshot below shows the all-clear state. **API**:

- `POST /api/admin/users/{id}/approve` (auto-grants the welcome 1M tokens)
- `POST /api/admin/users/{id}/reject`



Step 6 — Read current generation usage and cost

In-product

- **Sidebar → Analytics → Generation Stats** (`/admin/analytics/generation-stats`): completed count, failed count, success rate, completion-breakdown donut, daily generations, failures-by-stage bar.



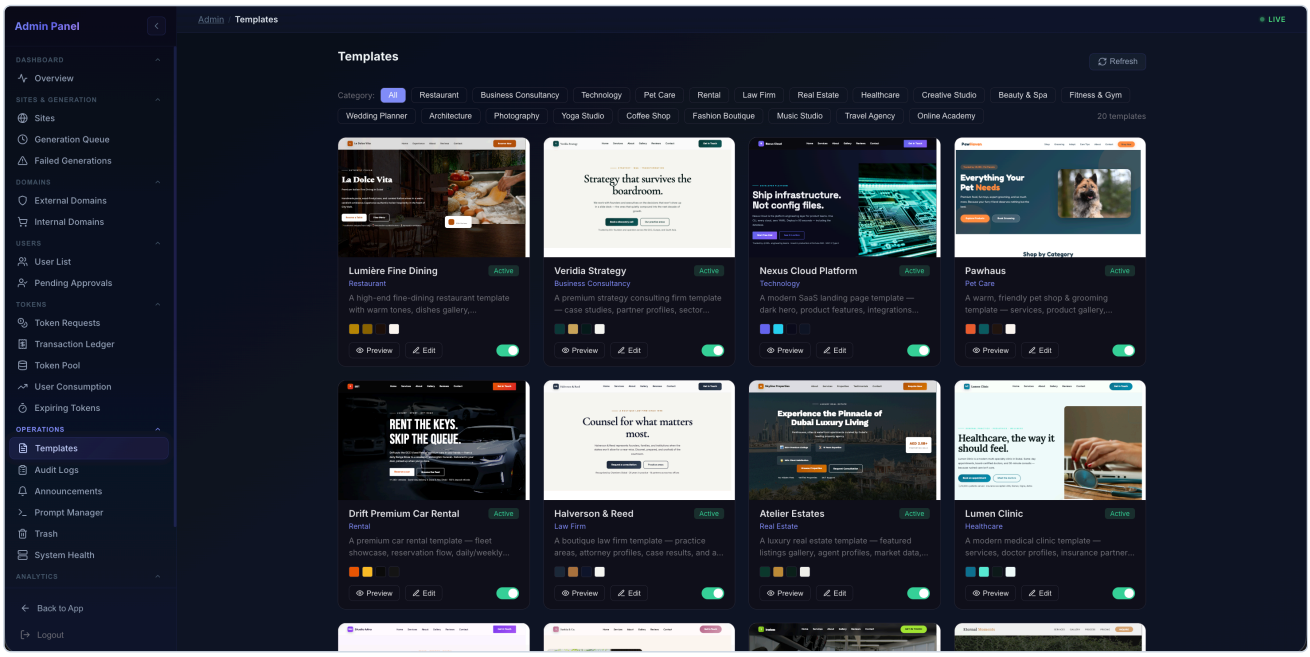
- **Sidebar → Analytics → Token Consumption** (`/admin/analytics/token-consumption`): consumed / granted / refunded / avg-daily-burn / utilisation, daily token activity, cumulative trend.



- **Sidebar → Analytics → Growth** (`/admin/analytics/growth`): total users, signups this week, active users, signups & active users over time, user-status donut.



- **Sidebar → Operations → Templates** (`/admin/operations/templates`): the public template gallery — toggle templates on/off across categories.

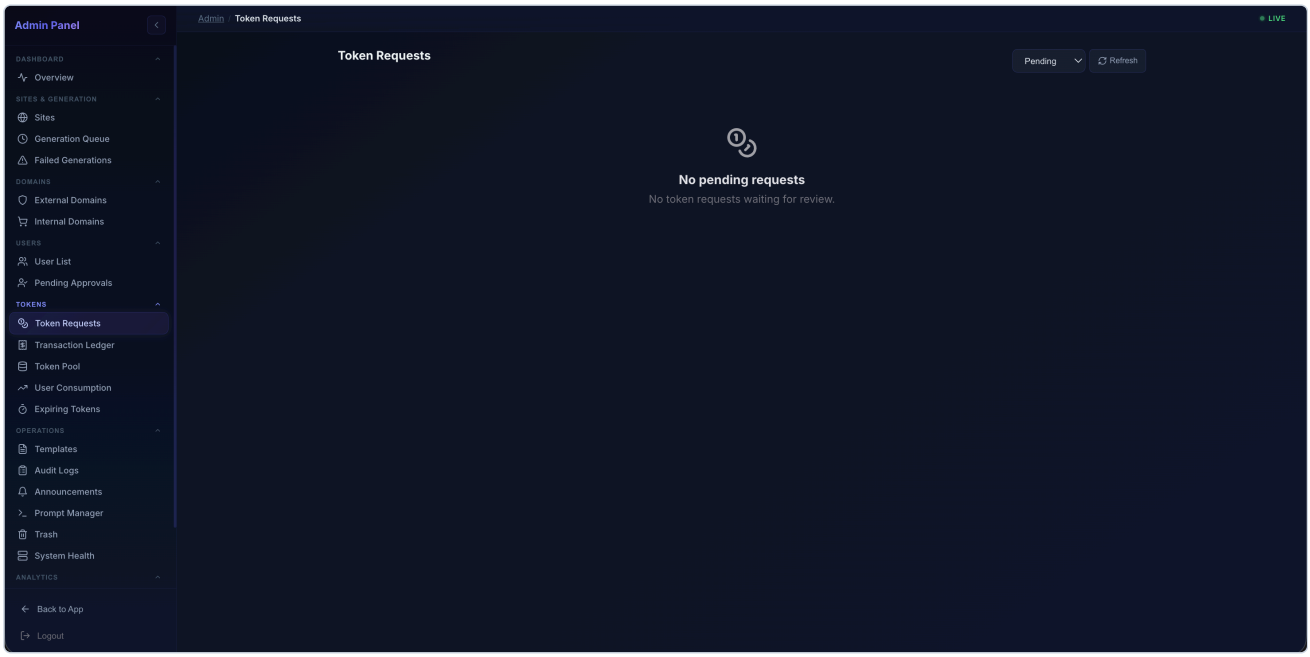


External (authoritative for billing)

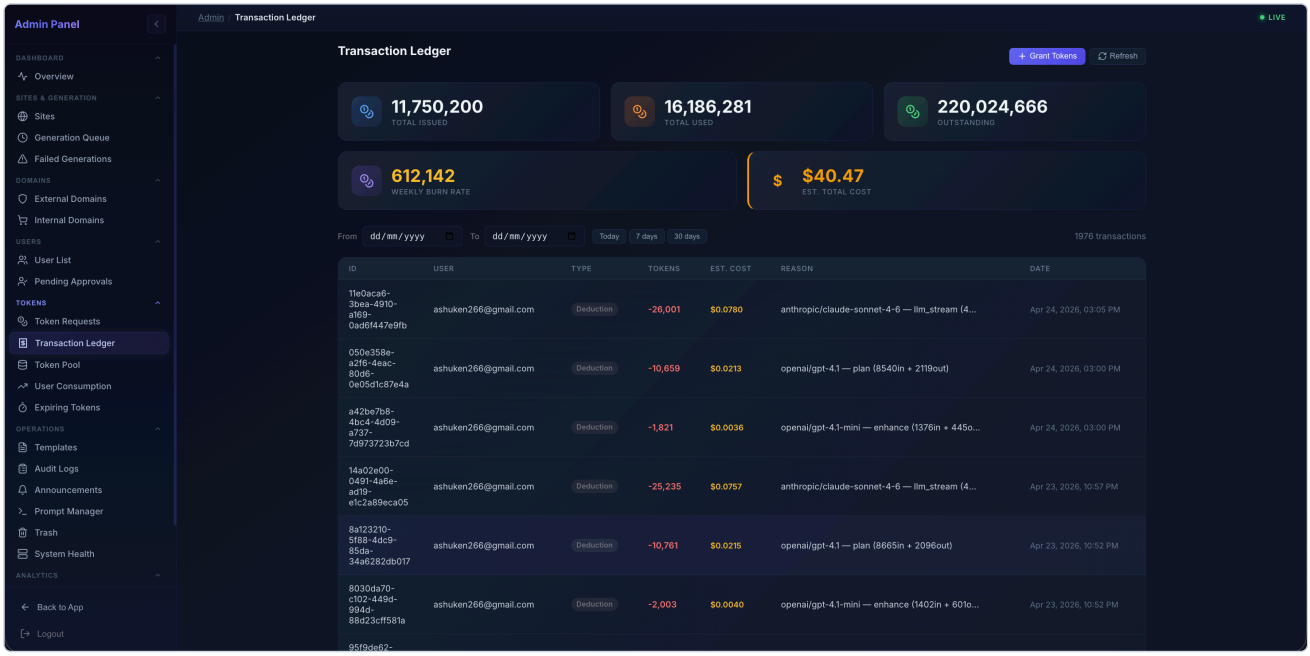
- **AWS Cost Explorer**, filtered by tag `Project=meydan-ai` , region `ap-southeast-1` .
- **OpenAI dashboard** under the production org — token usage and per-model spend.
- **Anthropic console** — same.
- **CloudWatch dashboard** in `ap-southeast-1` : `meydan-production` — request rate, p95, 5xx, ALB target health.

Step 7 — Manage tokens (request queue, ledger, pools)

- **Sidebar → Tokens → Token Requests** — pending user requests for top-ups; admin approves / rejects.

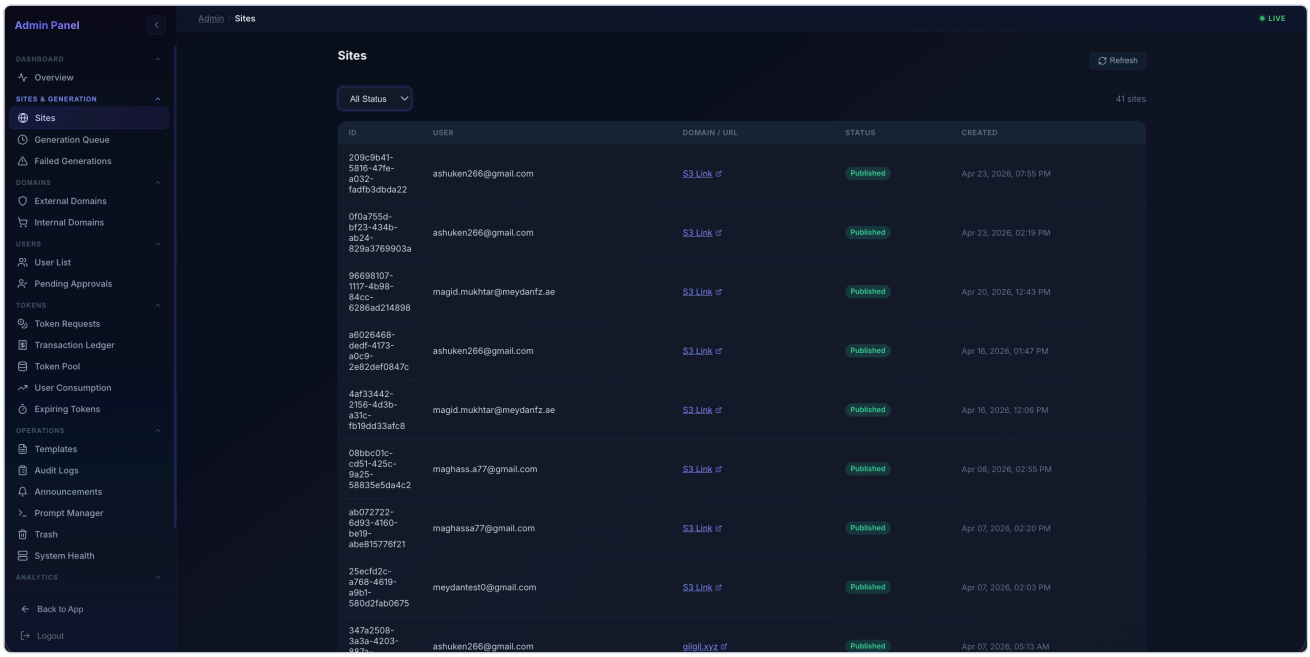


- **Sidebar → Tokens → Transaction Ledger** — every grant, consumption, refund. Searchable / filterable / CSV-exportable.

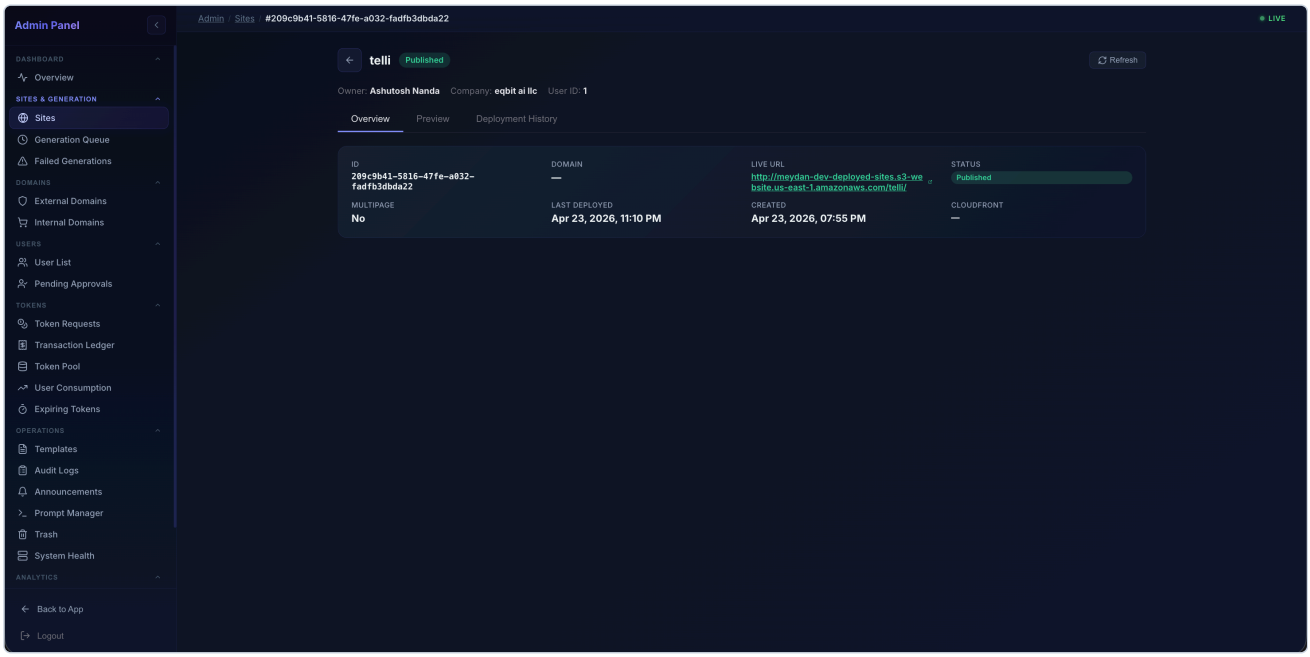


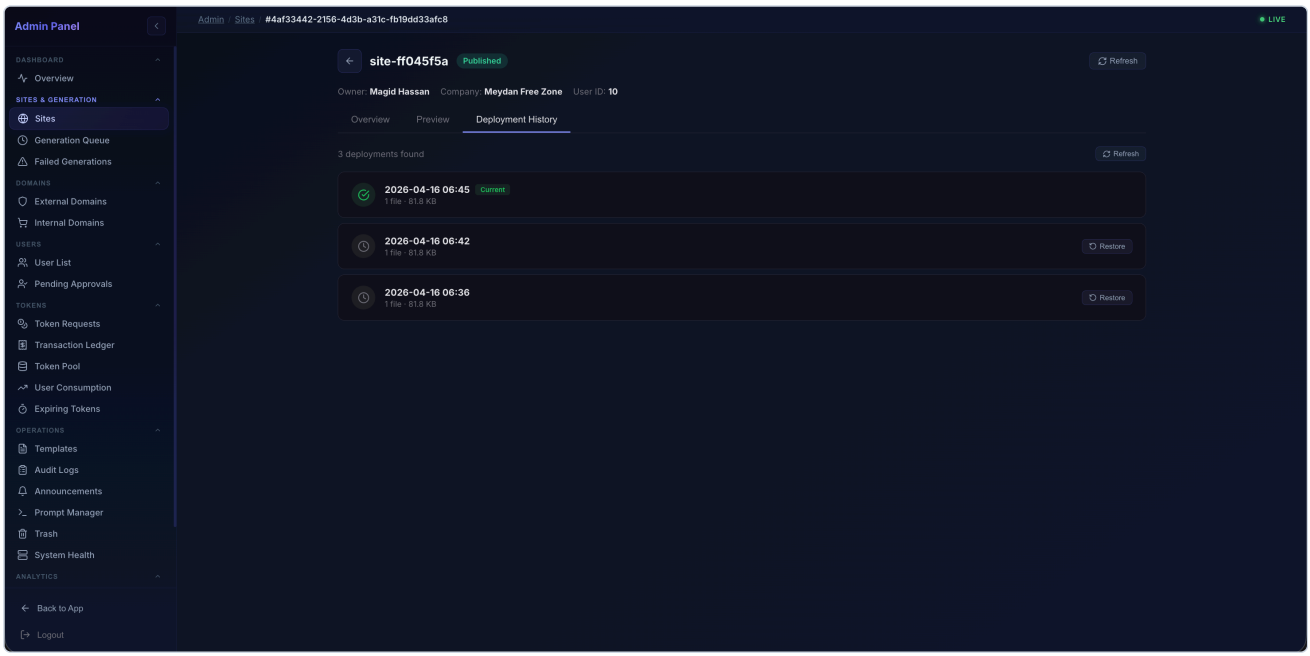
Step 8 — Manage sites & deployments

- **Sidebar → Sites & Generation → Sites** — every published site across all tenants.

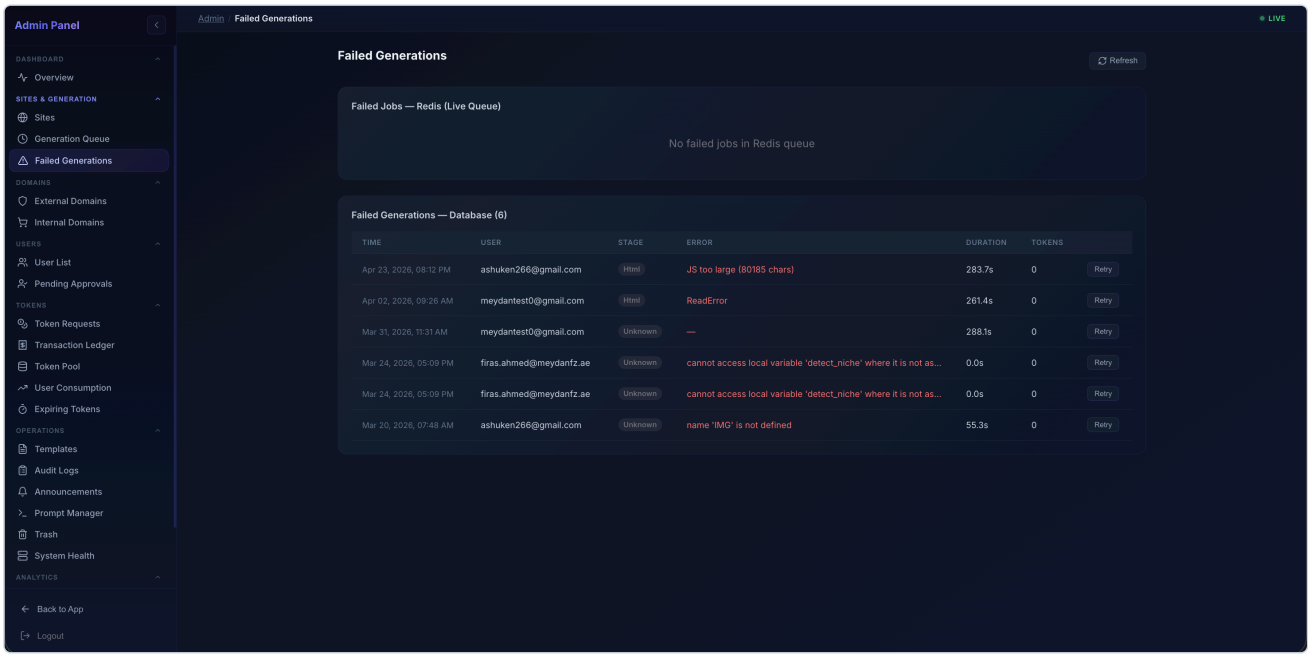


- Click into a site → tabs **Overview / Preview / Deployment History**:



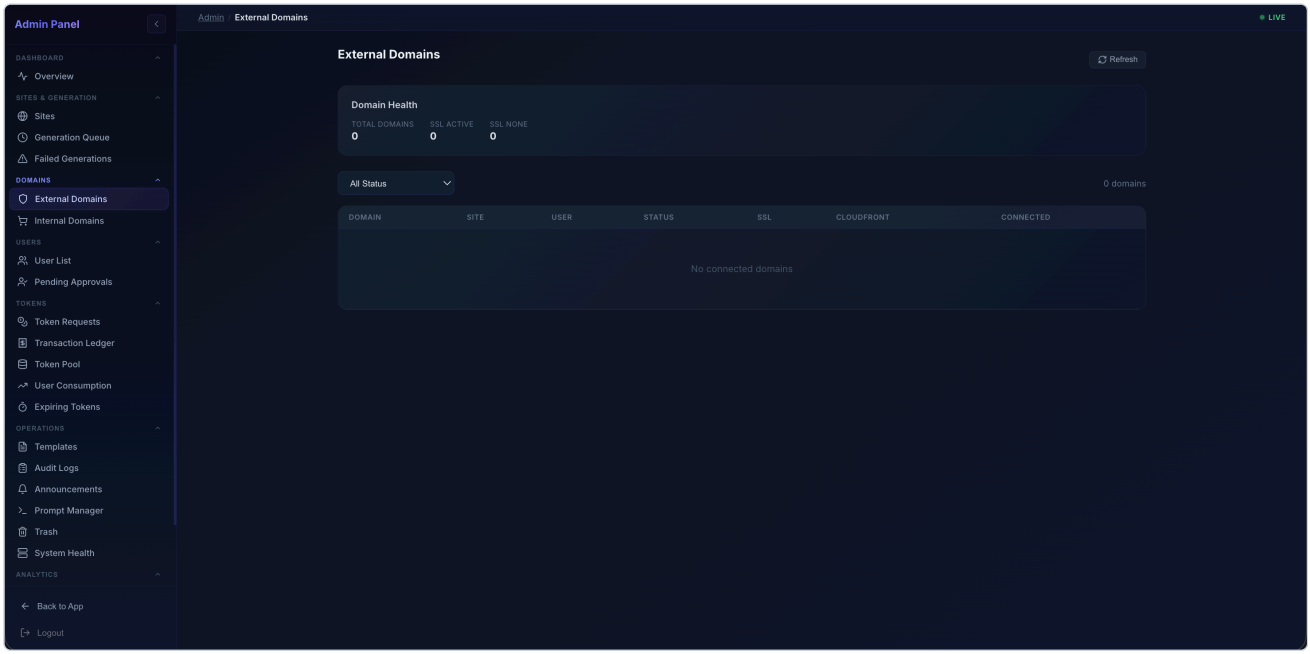


- **Sidebar → Sites & Generation → Failed Generations** — live Redis queue + DB-persisted failures, with retry buttons.



Step 9 — Manage domains

- **Sidebar → Domains → External Domains** — customer-purchased domains via OpenProvider. Status / verification / SSL state.



- **Sidebar → Domains → Internal Domains** — `*.meydanfzai.com` subdomains issued to tenants.

Step 10 — Change a prompt or model (Prompt Manager — NOT YET WIRED)

> ⚠️ **Important — Prompt Manager is UI-and-storage only at handover. It is not wired into the live generation pipeline and has not been end-to-end tested.** Publishing a new version writes a row to `prompt_versions` in the DB but the Worker still reads hardcoded prompts from `backend/app/utils/prompt_pipeline.py`. To actually change a prompt today, edit `prompt_pipeline.py`, commit, and push to `main`. See "Unfinished items" in `10_known_issues_and_support.md` — this is the top item.

What exists today (UI + DB only): **Sidebar → Operations → Prompt Manager** (`/admin/operations/prompt-manager`). You can pick a stage (Stage 0/1/2/3), browse the version history table, click **+ New Version**, paste the new system prompt, write a release note, **Publish**. The DB records the change. **Nothing in the live pipeline reads it.**

Endpoints that work (write to DB but don't affect generation):

- `GET /api/admin/prompts` — list stages and stored versions
- `POST /api/admin/prompts/{stage}/versions` — submit a new version
- `POST /api/admin/prompts/{stage}/versions/{id}/publish` — flag as active in DB
- `POST /api/admin/prompts/{stage}/versions/{id}/rollback` — flag previous as active in DB

To finish the wiring (~1 day of work):

1. In `backend/app/utils/prompt_pipeline.py`, replace the hardcoded prompt strings with a lookup function that reads the active version from `prompt_versions` and falls back to the in-file default if the DB has no published row.
2. Add a 60-second in-process cache so each generation request doesn't hit the DB.
3. Add a "preview generation with this draft prompt" admin endpoint so a new version can be smoke-tested before publish.
4. Cover with a unit test: publishing a known-good version routes the generation request through it; rolling back restores the previous prompt.

The model name is a metadata field on the version (already in the schema), so once wired the rollback restores both prompt text and model.

What to do RIGHT NOW to change a prompt

Until the wiring is done, treat prompts as code:

```
# 1. Edit the prompt
$EDITOR backend/app/utils/prompt_pipeline.py

# 2. Commit + push to dev to test
git add backend/app/utils/prompt_pipeline.py
git commit -m "prompts: tighten Stage 2 hero contrast rule"
git push origin dev

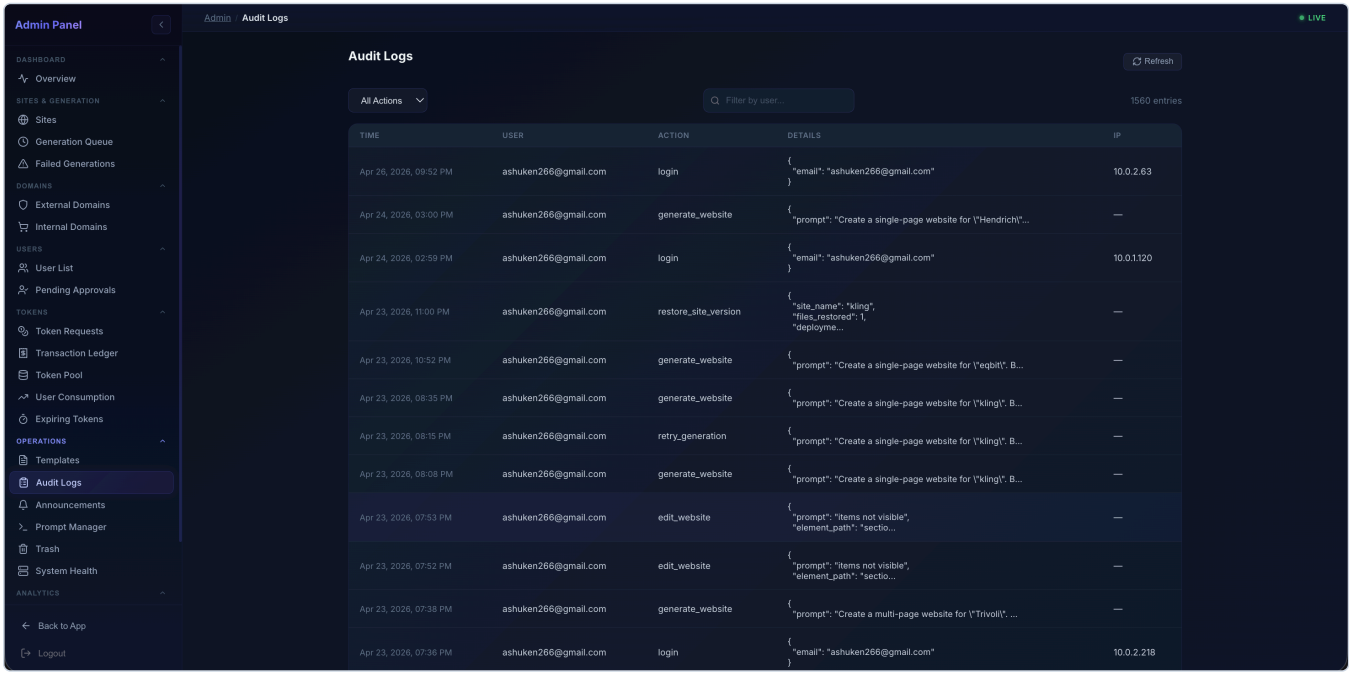
# 3. Verify on dev (https://dev.meydanfzai.com or the dev ALB DNS) — generate one site
# 4. If it looks right, merge to main
git checkout main && git merge dev && git push origin main
```

CI builds the new image and rolls the prod ECS services within ~5 minutes.

Step 11 — Read application logs and the audit trail

Audit trail (in-product)

Where: **Sidebar → Operations → Audit Logs**. Every admin action plus auth events. Filterable by user. **API**: `GET /api/admin/audit-log?from=&to=&q=&page=`.



Application logs (CloudWatch)

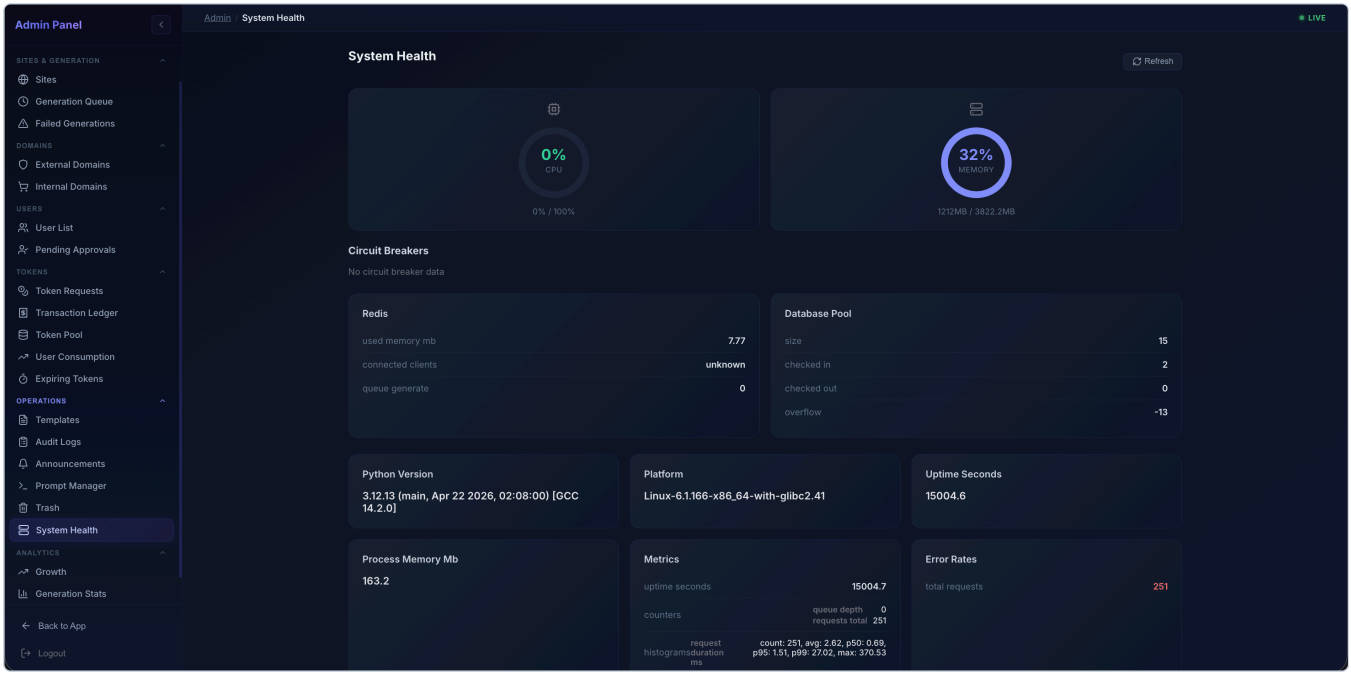
```
# Live tail (last 5 min, errors only)
aws logs tail /ecs/meydan-prod-api --since 5m \
  --filter-pattern '{ $.level = "ERROR" }' \
  --region ap-southeast-1 --profile prod

# Trace a specific user-reported issue (correlation ID is in the X-Correlation-ID response header)
aws logs tail /ecs/meydan-prod-api --since 30m \
  --filter-pattern '{ $.correlation_id = "0cf84c22-0c0f-47fe-b8e0-1ae8a667b7a3" }' \
  --region ap-southeast-1 --profile prod
```

Step 12 — Inspect system health

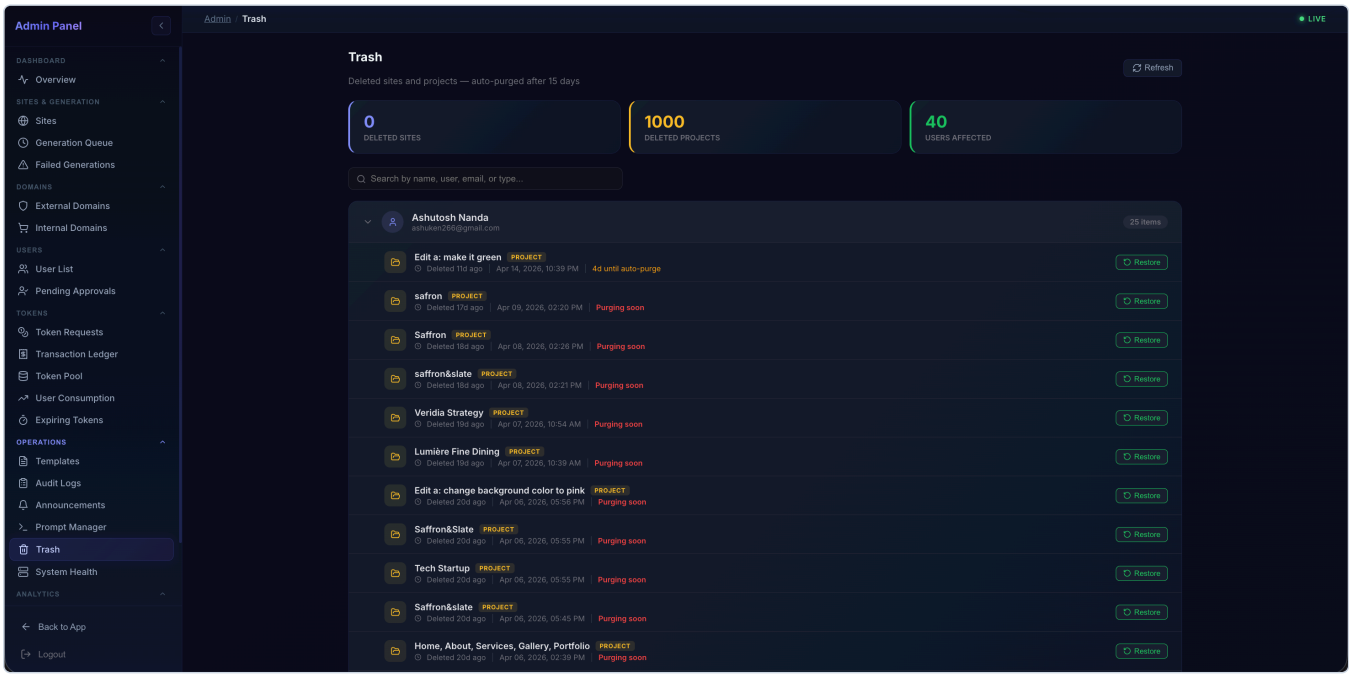
Where: **Sidebar → Operations → System Health**. **What it shows**: CPU, memory, disk, circuit breakers (Redis, Database Pool), Python / Linux version, uptime, process metrics, error rate, request count.

If a circuit breaker is open (e.g. OpenAI keeps timing out), reset with: `POST /api/admin/circuit/{openai|anthropic}/reset`.



Step 13 — Recover from accidents (Trash)

Where: **Sidebar → Operations → Trash**. Soft-deleted sites and projects, auto-purged after 15 days. Each row has **Restore**.



5. Code Structure

A map of the repository. Open this file before opening the code.

```
meydan-website-generator/
├── backend/                    Python · FastAPI · the brains.
│   ├── app/
│   │   ├── main.py             FastAPI app factory; middleware + router wiring.
│   │   ├── database.py         Async SQLAlchemy engine + tenant-schema dispatch.
│   │   ├── models/             SQLAlchemy ORM models (User, Site, EditLog, ...).
│   │   ├── schemas/            Pydantic request / response models.
│   │   ├── routes/             HTTP endpoints — one file per concern.
│   │   │   ├── auth.py             /api/auth/*  (signup, login, verify, reset)
│   │   │   ├── generator.py        /api/generator/*  (SSE generation pipeline)
│   │   │   ├── projects.py         /api/projects/*  (CRUD on saved sites)
│   │   │   ├── deploy.py           /api/deploy/*    (S3 + CloudFront + ACM)
│   │   │   ├── domains.py          /api/domains/*   (custom-domain purchase)
│   │   │   ├── admin/              /api/admin/*     (operator surface; sub-routers)
│   │   │   └── landing_chat.py     /api/landing/chat  (public FAQ chatbot)
│   │   ├── utils/              Cross-cutting helpers.
│   │   │   ├── llm_clients.py      AsyncAnthropic + AsyncOpenAI singletons; circuit breaker decorator.
│   │   │   ├── prompt_pipeline.py  Stage 0-3 prompts; the four-stage generation orchestration.
│   │   │   ├── content_moderation.py   Keyword + LLM moderation (fail-closed).
│   │   │   ├── input_sanitizer.py  XSS / SQLi / password-strength / email validation.
│   │   │   ├── s3_deploy.py        Customer-site upload + URL generation.
│   │   │   ├── cloudfront_acm.py   Custom domain SSL flow.
│   │   │   ├── email.py            Resend / SES provider abstraction.
│   │   │   └── storage.py          User uploads bucket access.
│   │   ├── middleware/         FastAPI middleware (correlation IDs, rate-limiting, security headers, request tracker).
│   │   ├── worker.py           Async worker process; consumes Redis queue, runs pipeline.
│   │   └── alembic/            Database migrations (one file per change).
│   ├── tests/                  pytest suites (unit + integration).
│   ├── Dockerfile              Multi-stage; final image runs gunicorn + UvicornWorker.
│   ├── entrypoint.sh           alembic upgrade head, then gunicorn.
│   └── .env.example            Every env var the backend reads.
│
├── frontend/                   React 18 · CRA · the user-facing UI.
│   ├── public/                 favicon, manifest, OG image, video poster.
│   ├── src/
│   │   ├── App.js              Route table; auth-guarded shell.
│   │   ├── App.css             Global dark-theme generator styles.
│   │   ├── meydan-theme.css    Light theme for landing / auth.
│   │   ├── config.js           apiUrl resolution (same-origin in prod, localhost in dev).
│   │   ├── components/
│   │   │   ├── Landing.jsx         Marketing page.
│   │   │   ├── Login.jsx + Signup.jsx
│   │   │   ├── Dashboard.jsx       Post-login site list.
│   │   │   ├── WebsiteGenerator.jsx  Editor workspace (chat + preview + Monaco).
│   │   │   ├── admin/              Admin SPA (one component per page; see /admin routes).
│   │   │   └── shared/             Modals, toasts, error banner, etc.
│   │   ├── services/
│   │   │   ├── api.js              Authenticated fetch wrapper; JWT in localStorage.
│   │   │   └── adminApi.js         Admin endpoint helpers.
│   │   └── hooks/              Reusable React hooks.
│   ├── nginx.conf              Production nginx config — serves /build with SPA fallback.
│   └── Dockerfile              Multi-stage (node build → nginx runtime).
│
├── infrastructure/             AWS infra-as-config (first-time provisioning).
│   ├── aws/
│   │   ├── cloudformation.yml      Parameterised stack template.
│   │   ├── ecs-task-api.json       Dev API task def (live source of truth lives in ECS).
│   │   ├── ecs-task-worker.json    Dev Worker task def.
│   │   ├── ecs-task-api-prod.json  Prod API task def template (Singapore).
│   │   ├── ecs-task-worker-prod.json  Prod Worker task def template.
│   │   ├── alarms.yml              CloudWatch alarms stack.
│   │   ├── waf.yml                 WAF rules stack.
│   │   ├── dashboard.json          CloudWatch dashboard JSON.
│   │   └── deploy.sh               One-shot bring-up script.
│   ├── migration/                  Helper scripts for cross-region data moves.
│   │   ├── pg_dump_us_east.sh
│   │   ├── pg_restore_singapore.sh
│   │   ├── s3_sync_uploads.sh
│   │   └── verify_migration.sh
│   └── README.md                   Pinned region + sizing decisions.
│
├── .github/workflows/          Real CI/CD.
│   ├── ci.yml                  Lint + test + security scan + frontend build + SonarCloud.
│   ├── deploy-dev.yml          Push to `dev` → us-east-1 (account 371170753756).
│   └── deploy-prod.yml         Push to `main` → ap-southeast-1 Singapore (account 290172088819).
```

```
│
├── docs/                      Long-form docs.
│   ├── Handover_Meydan_2026-04/    This bundle.
│   └── ecommerce-feature-plan.{md,pdf}  Architectural plan for the parked e-commerce feature.
│
├── nginx.conf                 Local-dev edge proxy (when running docker-compose).
├── docker-compose.yml         Local dev stack (PG + Redis + frontend + backend + worker).
├── sonar-project.properties   SonarCloud config.
└── README.md                  Top-level repo overview.
```

Where each concern lives

Concern	Path
Business logic	`backend/app/routes/` (one router per resource) + `backend/app/utils/prompt_pipeline.py` (generation orchestration)
Prompts	`backend/app/utils/prompt_pipeline.py` (defaults). Live overrides in DB table `prompt_versions` via the admin Prompt Manager.
Third-party clients	`backend/app/utils/llm_clients.py` (OpenAI, Anthropic), `backend/app/utils/email.py` (Resend / SES), `backend/app/utils/cloudfront_acm.py` (ACM, CloudFront), `backend/app/utils/s3_deploy.py` (S3)
Config	`backend/.env.example` (all env vars + comments), `backend/app/config.py` (typed settings), live ECS task definitions for prod values
Auth	`backend/app/routes/auth.py` (issue), `backend/app/middleware/` (verify), `backend/app/utils/jwt.py`
Tenant isolation	`backend/app/database.py::get_tenant_db` (sets `search_path`)

6. API Reference

Base URL: `https://meydanfzai.com/api`. Interactive Swagger at `/docs` (only in non-production `ENVIRONMENT` values).

Authentication

All non-public endpoints require a JWT in the `Authorization` header:

```
Authorization: Bearer <token>
```

Obtain a token via `POST /api/auth/login`. Token claims: `user_id`, `email`, `is_admin`, `license_number`, `db_schema`. Default expiry **24 hours** (`JWT_EXPIRE_MINUTES=1440`).

Common error codes

Status	Meaning
`400`	Validation failure — payload didn't match the schema. Body: `{"detail": "..."}`.
`401`	Missing / invalid / expired JWT. Body: `{"detail": "Authentication required"}`.
`403`	Authenticated but not authorised (e.g. non-admin hitting `/api/admin/*`, or trying to access another tenant's data).
`404`	Resource not found OR exists in another tenant (we deliberately conflate to prevent IDOR fishing).
`409`	Conflict — e.g. signup with an existing email; deploy slug collision.
`429`	Rate limited. `Retry-After` header tells you when.
`500`	Server error. Body: `{"detail": "Internal server error"}`. The full traceback is in CloudWatch keyed by the `X-Correlation-ID` header.
`503`	LLM circuit breaker open (provider failing). Retry after 60 s.

Rate limits

Sliding-window per `(user_id, route_class)` keyed in Redis.

Route class	Limit
`/api/auth/*`	10 req / 5 min per IP
`/api/landing/*`	30 req / 5 min per IP
`/api/generator/*`	20 req / hour per user
`/api/admin/*`	200 req / min per admin
All other	60 req / min per user

Endpoints

`POST /api/auth/signup`

Create a new account. Triggers email verification.

Request:

```
{
  "email": "founder@acme.com",
  "password": "ValidStrong#1Pass",
  "full_name": "Acme Founder",
  "company_name": "ACME Trading FZ-LLC",
  "license_number": "MZ-12345"
}
```

Response 201:

```
{ "user_id": 1429, "pending_approval": true, "email_verified": false }
```

Errors: `409` if email exists; `400` for validation (e.g. weak password — must be 12+ chars with upper, lower, digit, symbol).

`POST /api/auth/login`

Request:

```
{ "email": "founder@acme.com", "password": "ValidStrong#1Pass" }
```

Response 200:

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIs...",
  "token_type": "bearer",
  "expires_in": 86400,
  "user": { "id": 1429, "email": "founder@acme.com", "is_admin": false, "is_approved": true }
}
```

Errors: `401` with generic "Invalid credentials" (does not reveal whether email exists).

`POST /api/auth/forgot-password`

Request: `{ "email": "founder@acme.com" }` **Response 200**: `{"ok": true}` — always 200, regardless of whether the email exists (anti-enumeration).

`GET /api/auth/me`

Returns the JWT-authenticated user's record.

Response 200:

```
{ "id": 1429, "email": "founder@acme.com", "is_admin": false, "is_approved": true, "tokens_balance": 847200 }
```

`POST /api/generator/generate-stream`

Streams generation events back via Server-Sent Events. Long-lived connection (up to 300 s).

Request:

```
{ "niche": "trading", "prompt": "ACME Trading is an import-export business based in Dubai..." }
```

Response 200 (`text/event-stream`):

```
event: stage
data: {"stage": "enhance", "progress": 0.10}

event: stage
data: {"stage": "plan", "progress": 0.25}

...

event: complete
data: {"site_id": "uuid-...", "preview_url": "/editor/uuid-..."}
```

Errors: `400` if moderation rejects; `429` if rate-limited; `503` if LLM circuit open.

`POST /api/sections/{section_id}/edit`

Apply an LLM-driven edit to a single section. Returns the patched section HTML/CSS/JS.

Request: `{ "instruction": "make the hero text bigger and use brand red" }` **Response 200**: `{ "html": "...", "css": "...", "js": "...", "edit_log_id": 8821 }`

`POST /api/deploy`

Package the current draft and publish to S3.

Request: `{ "session_id": "uuid-..." }` **Response 200**:

```
{ "url": "https://acme-trading.s3-website.ap-southeast-1.amazonaws.com", "deployed_at": "2026-04-27T00:34:11Z" }
```

Errors: `403` if `session_id` doesn't belong to caller; `500` if upload fails (generic message; full traceback logged).

`POST /api/domains/buy`

Purchase a domain via OpenProvider, attach to the user's deployed site.

Request: `{ "domain": "acme-trading.com", "registration_years": 1 }` **Response 200**: `{ "domain": "acme-trading.com", "status": "pending_dns", "estimated_active_in_seconds": 600 }`

The flow is async — the DNS record (Route 53) is created, ACM cert is requested, the API returns immediately and the user is notified via email when the domain is active. ALB idle is 300 s, so the request must not block on DNS propagation; clients use the `purchaseDomainAndWait` helper that polls `GET /api/domains/{name}/status`.

`POST /api/landing/chat`

Public FAQ chatbot — no auth required. Anthropic Sonnet 4.6 backed.

Request:

```
{ "messages": [ { "role": "user", "content": "How long does generation take?" } ] }
```

Response 200: `{ "answer": "Generation takes about 60–90 seconds end-to-end..." }`

Rate-limited 30 req / 5 min / IP.

Admin endpoints (all require `is_admin=true`)

Method	Path	Purpose
`GET`	`/api/admin/overview`	Stats panels for the Overview page
`GET`	`/api/admin/users?q=&page=`	Paginated user search
`GET`	`/api/admin/users/{id}`	User detail
`POST`	`/api/admin/users/{id}/approve`	Flip `is_approved=true` and grant 1M tokens
`POST`	`/api/admin/users/{id}/reject`	Mark user rejected, send email
`POST`	`/api/admin/users/{id}/grant-tokens`	`{amount, reason}` — credits tokens
`POST`	`/api/admin/users/{id}/suspend`	Compliance hold
`GET`	`/api/admin/sites`	All deployed sites across tenants
`GET`	`/api/admin/generation-stats`	Token spend, latency by stage
`GET`	`/api/admin/prompts`	Prompt manager — list stages
`POST`	`/api/admin/prompts/{stage}/versions`	Submit new prompt version
`POST`	`/api/admin/prompts/{stage}/versions/{id}/publish`	Activate a version
`POST`	`/api/admin/prompts/{stage}/versions/{id}/rollback`	Revert to prior
`GET`	`/api/admin/audit-log?from=&to=&q=`	Audit trail (paginated, filterable)
`GET`	`/api/admin/system-health`	Circuit-breaker state, queue depth
`POST`	`/api/admin/circuit/{provider}/reset`	Force-close a circuit breaker

7. Deployment Runbook

Two flavours: **routine deploy** (push to a branch — CI does the rest) and **first-time provisioning** (clean cloud account → running production).

Routine deploy — push and CI does it

Branch	CI workflow	AWS account	Region
`dev`	`.github/workflows/deploy-dev.yml`	`371170753756`	`us-east-1`
`main`	`.github/workflows/deploy-prod.yml`	`290172088819`	`ap-southeast-1` (Singapore)

Each workflow:

1. Builds `meydan-api` and `meydan-frontend` images with `--platform linux/amd64`.
2. Pushes to ECR.
3. `aws ecs update-service --force-new-deployment` on each of the 3 services (api, worker, frontend).
4. `aws ecs wait services-stable` (up to ~10 min).

`entrypoint.sh` runs `alembic upgrade head` before gunicorn boots — migrations apply automatically.

> **Known annoyance**: the wait step occasionally times out on a successful deploy. If `/api/health` returns 200 within 5 min of the push, treat the deploy as green.

Prerequisites

Accounts

- AWS account with administrator access. Production region: `ap-southeast-1` (Singapore). Dev region: `us-east-1`.
- GitHub access to the private `meydan-website-generator` repo.
- AWS Route 53 hosted zone for `meydanfzai.com` (apex + wildcard records).
- OpenProvider reseller account.
- Resend account, `meydanfzai.com` domain verified (SPF + DKIM TXT records).
- OpenAI + Anthropic API keys (production tier — see `10_known_issues_and_support.md` for tier requirements at scale).

Local tools

- Docker Desktop (for `--platform linux/amd64` cross-build — **required**, not optional).
- AWS CLI v2.
- GitHub CLI (optional).
- Node 20+, Python 3.12, `psql` 15 client.

Secrets

All listed in `08_environment_and_secrets.md`.

First-time provisioning — clean account → running prod

> Throughout, set `export AWS_REGION=ap-southeast-1 && export AWS_PROFILE=prod`.

1. ECR + Secrets Manager (free / near-free)

```
aws ecr create-repository --repository-name meydan-api --image-scanning-configuration scanOnPush=true
aws ecr create-repository --repository-name meydan-frontend --image-scanning-configuration scanOnPush=true

# Populate Secrets Manager — see 08_environment_and_secrets.md for the full list.
# For each secret, IMPORTANT: strip trailing whitespace before storing or the value
# becomes an Illegal HTTP header value when injected as env var into ECS.
for s in jwt-secret openai-api-key anthropic-api-key resend-api-key openprovider-username \
         openprovider-password openprovider-reseller-id pexels-api-key unsplash-access-key \
         gemini-api-key; do
  read -p "Value for meydan/prod/$s: " v
  printf '%s' "$v" > /tmp/sec.txt   # printf, not echo — no trailing newline
  aws secretsmanager create-secret --name "meydan/prod/$s" --secret-string "file:///tmp/sec.txt"
done
shred -u /tmp/sec.txt
```

2. Network (VPC + subnets + IGW + SGs)

```
VPC_ID=$(aws ec2 create-vpc --cidr-block 10.10.0.0/16 \
  --tag-specifications 'ResourceType=vpc,Tags=[{Key=Name,Value=meydan-prod-vpc}]' \
  --query 'Vpc.VpcId' --output text)
aws ec2 modify-vpc-attribute --vpc-id $VPC_ID --enable-dns-hostnames
aws ec2 modify-vpc-attribute --vpc-id $VPC_ID --enable-dns-support

# Public subnets in 3 AZs
for i in a b c; do
  aws ec2 create-subnet --vpc-id $VPC_ID --cidr-block 10.10.$([ $i = a ] && echo 1 || ([ $i = b ] && echo 2 || echo 3)).0/24 \
    --availability-zone ap-southeast-1$i \
    --tag-specifications "ResourceType=subnet,Tags=[{Key=Name,Value=meydan-prod-subnet-1$i}]"
done

# IGW, route table, security groups — see infrastructure/aws/cloudformation.yml for the parameterised version.
```

3. RDS (PostgreSQL 15 Multi-AZ)

```
DB_PASS=$(openssl rand -base64 32 | tr -d '+/=' | head -c 24)
aws rds create-db-instance \
  --db-instance-identifier meydan-prod-db \
  --db-instance-class db.t4g.medium \
  --engine postgres --engine-version 15.17 \
  --master-username meydan --master-user-password "$DB_PASS" \
  --allocated-storage 20 --max-allocated-storage 100 \
  --storage-type gp3 --storage-encrypted \
  --multi-az \
  --vpc-security-group-ids "$DB_SG" \
  --db-subnet-group-name meydan-prod-db-subnet \
  --backup-retention-period 14 \
  --deletion-protection \
  --db-name meydan_db
# Wait ~12 min until status=available, then write the URL into Secrets Manager.
```

4. ElastiCache Redis

```
aws elasticache create-cache-cluster \
  --cache-cluster-id meydan-prod-redis \
  --engine redis --cache-node-type cache.t3.small --num-cache-nodes 1 \
  --cache-subnet-group-name meydan-prod-redis-subnet \
  --security-group-ids "$REDIS_SG" \
  --snapshot-retention-limit 7
```

5. ALB + target groups + ACM cert

```
ALB_ARN=$(aws elbv2 create-load-balancer --name meydan-prod-alb --type application \
  --subnets $SUBNET_1A $SUBNET_1B $SUBNET_1C --security-groups $ALB_SG \
  --query 'LoadBalancers[0].LoadBalancerArn' --output text)
aws elbv2 modify-load-balancer-attributes --load-balancer-arn $ALB_ARN \
  --attributes Key=idle_timeout.timeout_seconds,Value=300

# Target groups: api on 8000, frontend on 3000. API uses lb_cookie stickiness for SSE.
# HTTPS listener on 443 with ACM cert; HTTP 80 redirects 301 to HTTPS.
# /api/* rule → API target group at priority 10; default → frontend.

# ACM cert with DNS validation:
aws acm request-certificate --domain-name meydanfzai.com \
  --subject-alternative-names "*.meydanfzai.com" --validation-method DNS
# Add the validation CNAME to Route 53 (or whichever DNS authority holds the zone).
```

6. ECS cluster + task definitions + services

```
aws ecs create-cluster --cluster-name meydan-prod-cluster \
  --capacity-providers FARGATE --settings name=containerInsights,value=enabled

aws ecs register-task-definition --cli-input-json file://infrastructure/aws/ecs-task-api-prod.json
aws ecs register-task-definition --cli-input-json file://infrastructure/aws/ecs-task-worker-prod.json
# Frontend task def is similar but image=meydan-frontend, port=3000, no secrets.

# Services — public subnets with assignPublicIp=ENABLED (no NAT Gateway, mirrors dev pattern).
aws ecs create-service --cluster meydan-prod-cluster \
  --service-name meydan-prod-api --task-definition meydan-prod-api \
  --desired-count 0 --launch-type FARGATE \
  --network-configuration "awsvpcConfiguration={subnets=[...],securityGroups=[...],assignPublicIp=ENABLED}" \
  --load-balancers "targetGroupArn=$API_TG,containerName=meydan-api,containerPort=8000" \
  --health-check-grace-period-seconds 120

# Repeat for worker (no LB) and frontend.
```

7. Autoscaling

```
# API: 2 → 6 on CPU 60%; Worker: 1 → 8 on CPU 50%; Frontend: 2 → 4 on CPU 60%.
aws application-autoscaling register-scalable-target \
  --service-namespace ecs \
  --resource-id service/meydan-prod-cluster/meydan-prod-api \
  --scalable-dimension ecs:service:DesiredCount \
  --min-capacity 2 --max-capacity 6
aws application-autoscaling put-scaling-policy \
  --service-namespace ecs --resource-id service/meydan-prod-cluster/meydan-prod-api \
  --scalable-dimension ecs:service:DesiredCount --policy-name api-cpu-target \
  --policy-type TargetTrackingScaling \
  --target-tracking-scaling-policy-configuration '{"TargetValue":60.0,"PredefinedMetricSpecification":{"PredefinedMetricType":"ECSSe
```

8. Data migration (only if migrating from an existing dev environment)

See `infrastructure/migration/` helper scripts:

- `pg_dump_us_east.sh` — dump dev DB
- `pg_restore_singapore.sh` — restore (drops and recreates `public` schema first to avoid the `schema "public" already exists` error from a fresh restore)
- `s3_sync_uploads.sh` — sync customer-uploaded assets cross-region
- `verify_migration.sh` — diff row counts between dev and prod

9. DNS cutover (CloudFront)

```
apex (meydanfzai.com)  A   →  CNAME  →  meydan-prod-alb-XXXX.ap-southeast-1.elb.amazonaws.com  (proxied / orange-cloud)
www                    A   →  CNAME  →  same
dev                    CNAME unchanged (still points to us-east-1 ALB for testing)
```

CloudFront's CNAME-flattening handles the apex CNAME natively. Set `proxied=true` to also get DDoS + WAF + edge caching.

10. WAF + CloudWatch + alarms

Apply `infrastructure/aws/waf.yml`, `infrastructure/aws/alarms.yml`, `infrastructure/aws/dashboard.json` via CloudFormation / AWS CLI.

Post-deployment checks

- `curl https://meydanfzai.com/api/health` returns `{"status":"ok","service":"meydan-website-generator"}`.
- `curl https://meydanfzai.com/` returns 200 with `Content-Type: text/html` and the React shell HTML.
- Sign up a throwaway user, generate one site, deploy it. The deployed site loads from `https://<slug>.s3-website.ap-southeast-1.amazonaws.com`.
- CloudWatch dashboard `meydan-production` panels green for 30 minutes.

Rollback

For a bad code deploy:

```
# Find the previous task def revision
aws ecs describe-services --cluster meydan-prod-cluster --services meydan-prod-api \
  --query 'services[0].deployments[].{rev:taskDefinition,status:status}'

# Roll the API back
aws ecs update-service --cluster meydan-prod-cluster --service meydan-prod-api \
  --task-definition meydan-prod-api:N-1 --force-new-deployment
```

For a bad data state — restore from the latest RDS snapshot (snapshot-restore creates a new instance; you then point Secrets Manager `database-url` at the new endpoint and force-redeploy).

Local setup — under 30 minutes for a new developer

```
git clone <repo>
cd meydan-website-generator

# 1. Backend
cd backend
python3.12 -m venv venv && source venv/bin/activate
pip install -r requirements.txt
cp .env.example .env   # fill in DATABASE_URL, OPENAI_API_KEY, ANTHROPIC_API_KEY at minimum
docker run -d --name meydan-pg -p 5432:5432 -e POSTGRES_USER=meydan -e POSTGRES_PASSWORD=meydan -e POSTGRES_DB=meydan_db postgres:15
docker run -d --name meydan-redis -p 6379:6379 redis:7
alembic upgrade head
uvicorn app.main:app --reload --port 8000   # API on http://localhost:8000

# 2. Worker (separate terminal)
cd backend && source venv/bin/activate
python -m app.worker

# 3. Frontend (separate terminal)
cd ../frontend
npm install
npm start   # http://localhost:3000

# Or skip the manual setup and run everything via docker-compose:
docker compose up
```

8. Environment and Secrets

Every environment variable the product reads. Plain values are set in the ECS task definition `environment` array; sensitive values are pulled at task-start from AWS Secrets Manager via `valueFrom` ARNs.

Source of truth

- **Live source**: the running ECS task definition. Inspect with:

  ```
  aws ecs describe-task-definition --task-definition meydan-prod-api \
    --region ap-southeast-1 --profile prod \
    --query 'taskDefinition.containerDefinitions[0].{env:environment,secrets:secrets[].name}'
  ```

- **Documented baseline**: `backend/.env.example` (every var with example value and one-line comment).

Environment variable table

Name	Purpose	Example value	Source
`ENVIRONMENT`	`production` / `staging` / `dev`. Gates Swagger UI and some defaults.	`production`	ECS env (plain)
`AWS_REGION`	Default region for boto3 clients.	`ap-southeast-1`	ECS env (plain)
`AWS_S3_DEPLOY_BUCKET`	Bucket for customer-deployed sites.	`meydan-prod-sites-sg`	ECS env (plain)
`API_BASE_URL`	Used in email links and worker → API callbacks.	`https://meydanfzai.com`	ECS env (plain)

Name	Purpose	Example value	Source
`FRONTEND_URL`	Used in email templates and CORS.	`https://meydanfzai.com`	ECS env (plain)
`ALLOWED_ORIGINS`	CORS whitelist (comma-separated).	`https://meydanfzai.com,https://www.meydanfzai.com`	ECS env (plain)
`JWT_ALGORITHM`	JWT signing alg.	`HS256`	ECS env (plain)
`JWT_EXPIRE_MINUTES`	Token lifetime.	`1440` (24 h)	ECS env (plain)
`JWT_SECRET`	HMAC key for signing. **Must match across all tasks/regions for session continuity.**	`(64 hex chars)`	Secrets Manager `meydan/prod/jwt-secret`
`DB_POOL_SIZE`	SQLAlchemy connection pool per worker process.	`15`	ECS env (plain)
`DB_MAX_OVERFLOW`	Burst capacity above pool size.	`25`	ECS env (plain)
`DATABASE_URL`	Async PG DSN.	`postgresql+asyncpg://meydan:…@meydan-prod-db.…rds.amazonaws.com:5432/meydan_db`	Secrets Manager `meydan/prod/database-url`
`REDIS_URL`	Redis connection.	`redis://meydan-prod-redis.…cache.amazonaws.com:6379/0`	Secrets Manager `meydan/prod/redis-url`
`DEV_MODE`	Enables generous debug endpoints + verbose logs. **Must be `false` in prod.**	`false`	ECS env (plain)
`LOG_LEVEL`	Python logger level.	`INFO`	ECS env (plain)
`TRUSTED_PROXY`	Trust `X-Forwarded-*` headers (set true behind ALB).	`true`	ECS env (plain)
`MODERATION_LLM_ENABLED`	Enable LLM-based pre-generation moderation. Flip to `false` in an OpenAI outage to fall back to keyword-only.	`true`	ECS env (plain)
`EMAIL_PROVIDER`	`resend` (default) or `ses`.	`resend`	ECS env (plain)

Name	Purpose	Example value	Source
`EMAIL_SENDER`	From-address.	`noreply@meydanfzai.com`	ECS env (plain)
`RESEND_SENDER_EMAIL`	Verified Resend sender.	`noreply@meydanfzai.com`	ECS env (plain)
`RESEND_API_KEY`	Resend API key.	`re_…`	Secrets Manager `meydan/prod/resend-api-key`
`OPENPROVIDER_API_URL`	Domain registrar base URL.	`https://api.openprovider.eu/v1beta`	ECS env (plain)
`OPENPROVIDER_USERNAME`	OpenProvider login.	(email)	Secrets Manager `meydan/prod/openprovider-username`
`OPENPROVIDER_PASSWORD`	OpenProvider password.	`…`	Secrets Manager `meydan/prod/openprovider-password`
`OPENPROVIDER_RESELLER_ID`	OpenProvider reseller account ID.	`…`	Secrets Manager `meydan/prod/openprovider-reseller-id`
`ANTHROPIC_API_KEY`	Anthropic API key (Stage 2 + 3).	`sk-ant-…`	Secrets Manager `meydan/prod/anthropic-api-key`
`OPENAI_API_KEY`	OpenAI API key (Stage 0 + 1, moderation).	`sk-proj-…`	Secrets Manager `meydan/prod/openai-api-key`
`GEMINI_API_KEY`	Optional fallback model (currently unused).	`AIza…`	Secrets Manager `meydan/prod/gemini-api-key`
`PEXELS_API_KEY`	Pexels stock-photo API.	`…`	Secrets Manager `meydan/prod/pexels-api-key`
`UNSPLASH_ACCESS_KEY`	Unsplash stock-photo API.	`…`	Secrets Manager `meydan/prod/unsplash-access-key`
`WORKER_CONCURRENCY`	Concurrent generation jobs per worker task.	`5`	Worker ECS env (plain)
`PORT`	API listen port.	`8000`	API ECS env (plain)
`WORKERS`	Number of gunicorn workers per task.	`8`	API ECS env (plain)
`SENTRY_DSN`	Optional — error tracking.	`https://…@sentry.io/…`	Secrets Manager `meydan/prod/sentry-dsn` (optional)
`AWS_ACCOUNT_ID`	Used for ARN generation.	`290172088819`	ECS env (plain)

Critical: trim trailing whitespace before storing in Secrets Manager

When creating a Secrets Manager entry from a CLI redirect (e.g. `aws … --output text > file && aws secretsmanager create-secret --secret-string file://file`), the `--output text` step appends a `\n`. ECS injects whatever bytes Secrets Manager has — newlines and all — into the env var. The `httpx` library (inside the OpenAI / Anthropic Python SDKs) then rejects header values containing newlines as `LocalProtocolError: Illegal header value`.

Always trim before storing:

```
printf '%s' "$VALUE" > /tmp/sec.txt        # printf, not echo
aws secretsmanager put-secret-value --secret-id meydan/prod/openai-api-key \
  --secret-string "file:///tmp/sec.txt"
```

This bit us during the Singapore migration — after fixing all 12 secrets, generation calls started working.

Rotation schedule

Secret	Rotate every	Procedure
`JWT_SECRET`	12 months OR after suspected leak	Rolling rotation requires keeping both old + new keys briefly so existing sessions don't all invalidate. Implementation pending — currently a hard rotation forces re-login.
`OPENAI_API_KEY`, `ANTHROPIC_API_KEY`	6 months	Generate new in provider console → update Secrets Manager → force-redeploy ECS → revoke old key in provider console.
`RESEND_API_KEY`	6 months	Same flow on resend.com.
`OPENPROVIDER_PASSWORD`	12 months	OpenProvider dashboard.
`DB master password`	12 months OR upon staff change	Rotate via `aws rds modify-db-instance --master-user-password` → update `meydan/prod/database-url` Secret with new value → force-redeploy.
`AWS IAM access keys`	6 months OR upon staff change	IAM console → create new key → update GitHub Actions secrets `AWS_ACCESS_KEY_ID_PROD` / `AWS_SECRET_ACCESS_KEY_PROD` → deactivate old.

Secret-leak audit

Run this to confirm no secret values are committed to the repository:

```
# Scan full git history for AWS-style keys, Anthropic / OpenAI style, JWT secrets
git log --all -p | grep -E "AKIA[A-Z0-9]{16}|sk-ant-api[0-9]{2}|sk-proj-|JWT_SECRET\s*=\s*['\"][a-f0-9]{32,}" | head
git log --all -p -S "ANTHROPIC_API_KEY=" | head
git log --all -p -S "OPENAI_API_KEY=" | head
```

Result at handover: **no production-key values appear in committed source**. The only key-shaped strings are placeholder values in `backend/.env.example` (`sk-ant-…`, `sk-proj-…`, `re_…`).

> A previous session (2026-04-23) noted dev IAM access keys had been pasted in chat. Those keys have been rotated as part of the Singapore migration; the value pasted in chat is no longer valid against any AWS account.

9. Ongoing Costs

> **Authoritative source**: the **Ongoing Costs sheet** in the handover workbook. The numbers below are engineering estimates. Update the workbook from AWS Cost Explorer and the provider billing pages on the day of delivery.

Snapshot — what's live and billing today

Production (`ap-southeast-1` , account `290172088819`)

Resource	Sizing	$ / month
RDS PostgreSQL `db.t4g.medium` Multi-AZ + 20 GB gp3	2 vCPU, 4 GB	~$140
ElastiCache Redis `cache.t3.small`	1 node	~$30
ALB	+ LCU + WAF rules	~$25
ECS Fargate API — 2 tasks min, autoscale 6 max (2 vCPU / 4 GB)	$83 / task / mo at min	$166 idle → $498 peak
ECS Fargate Worker — 1 task min, autoscale 8 max (2 vCPU / 4 GB)	$83 / task / mo	$83 idle → $663 peak
ECS Fargate Frontend — 2 tasks fixed (1 vCPU / 2 GB)	$42 / task / mo	$84
WAF	rules + WCU	~$10
CloudWatch logs + alarms + dashboard		~$20
Secrets Manager (12 secrets)	$0.40 / secret	$5
ECR + S3 + small misc		~$5
Idle (no traffic)		**~$570**
Peak (200 concurrent generations sustained)		**~$1,520**
Realistic monthly bill (B2B SaaS pattern, 4–6 hr active per day)		**~$700 – $900**

Dev (`us-east-1` , account `371170753756`) — testing only post-migration

Resource	Sizing	$ / month
RDS PostgreSQL `db.t3.medium` single-AZ	2 vCPU, 4 GB	~$60
ElastiCache Redis `cache.t3.small`	1 node	~$24
ALB		~$16
ECS Fargate API + Worker + Frontend	fixed counts	~$70
Subtotal — running 24/7		**~$170**

> **Optimisation**: the user has indicated dev should be scaled to 0 between testing sessions. If that lands, dev burn drops to ~30/*month*(*RDSstorage* + *Redisidle* + *ALB*), *regaining* 140 / month.

Third-party services

Service	Plan	Monthly	Billing cycle	Notes
OpenAI	(production tier)	**$300 – $5,000**	Monthly post-pay	At observed rate (~$0.04 per generation for stages 0+1), 1,000 sites/mo ≈ $40; 50,000 sites/mo ≈ $2,000. **Tier upgrades are critical-path for scale** — see `10_known_issues_and_support.md` .
Anthropic	(production tier)	**$1,000 – $12,000**	Monthly post-pay	Stages 2+3 dominate cost (~$0.21 per generation). Same tier-upgrade caveat.
Resend	(Transactional)	$20 – $100	Monthly	Welcome emails, password reset, generation notifications, custom-domain alerts.
OpenProvider	(Domain reseller)	$0 fixed + per-domain	Per-purchase	Each customer-purchased domain (~$10 – $25 / year retail margin). Pass-through cost.
AWS CloudFront	Pay-as-you-go	$5 – $50	Monthly	First 1 TB egress / month free; thereafter ~$0.085 / GB in ap-southeast-1. Per-customer-domain distributions add $0 fixed (request-priced). One distribution per custom domain currently.

Service	Plan	Monthly	Billing cycle	Notes
Pexels API	Free	$0	—	Stock photos for generated sites.
Unsplash API	Free	$0	—	Same.
Sentry (optional)	Developer	$0 – $26	Monthly	Error tracking. Currently disabled — `SENTRY_DSN` is unset.
SonarCloud	Open-source	$0	—	Code quality scans — currently red on every push because `SONAR_TOKEN` is missing from GitHub repo secrets. Fix or remove the SonarCloud step from `ci.yml`.

Cost-per-active-user estimate

Assumptions:

- Each *active user* generates 3 sites per month (typical SMB onboarding).
- Each generation costs ~$0.25 in LLM tokens (Stage 0–3 combined).
- Each generation hits the API ~30 times during the editing session (CRUD, edit-stream, deploy).
- AWS infrastructure cost is amortised across all active users at a given month.

Per-active-user cost:

- LLM: 3 generations × 0.25 = **0.75**
- AWS variable (extra ECS task-time, S3 puts, CloudWatch ingestion): **~$0.10**
- Email + DNS + misc: **~$0.05**
- **Variable cost per active user: ~$0.90 / month**
- **Fixed cost**: $700 – 900*monthly*.*Per* – *userfixedcostfallsfastwithscale*(*e*.*g*.1, 000*activeusers*→ 0.80 / user fixed; 10,000 active users → ~$0.08 / user fixed).
- **All-in per-active-user at 1,000 active users / month: ~$1.70**.

Free tiers, trial periods, and cutover dates

Service	Free tier ends when	Action when it ends
AWS new-account credits	The prod account `290172088819` does not currently have promotional credits applied; if the client transfers AWS credits, apply via Billing → Credits before regular charges land.	N/A unless transferred.
AWS CloudFront	First 1 TB egress / month is free permanently. Above that, ~$0.085 / GB in ap-southeast-1.	Watch the egress trend in CloudFront → Reports → Usage.
Sentry Developer plan	At 5,000 errors / month — automatic upgrade prompt.	Currently disabled; no immediate concern.
Pexels / Unsplash	Soft-rate-limited, never billed at our scale.	—
Resend	First 3,000 emails / month free, then $20 / mo for 50,000.	Already billing — verify in workbook.
OpenAI / Anthropic	Pay-as-you-go from token #1; no free tier.	Already billing.

Where to look on day-of-delivery

Question	Where
"What did AWS bill us last month?"	AWS Cost Explorer, filter `Environment=production` and `Project=meydan-ai`.
"What did OpenAI bill us last month?"	platform.openai.com → Usage → Costs.
"What did Anthropic bill us last month?"	console.anthropic.com → Plans & billing.
"What's our run-rate this month?"	Combine the three above; the **Ongoing Costs sheet** in the workbook keeps the historical roll-up.

10. Known Issues and Support

Everything not finished, and everyone who still needs to be called when things break.

Open bugs

#	Title	Severity	Status	Steps to reproduce	Workaround
1	Carousels / sliders do not function in generated sites	Medium	**CLOSED** (2026-04-27)	Carousel section now ships a known-good helper script that wires prev / next + autoplay against the standardised section markup. Verified end-to-end in dev.	—
2	LinkedIn iframes 404 / X-Frame-Options reject	Low	**CLOSED** (2026-04-27)	Stage 2 prompt updated to never emit iframes pointing at social-media platforms; `s3_deploy.py` post-processes any leftover social iframes into branded link cards.	—
3	Duplicate CSS / JS in some generated sites	Low	**CLOSED** (2026-04-27)	Deploy pipeline now runs a dedup pass before upload; animation-library snippet is included once at the bottom. Deployed-site bundle size dropped ~5–8%.	—
4	LLM occasionally generates dark hero with dark text → invisible	Medium	**CLOSED** (2026-04-27)	Stage 2 prompt tightened with explicit contrast rules; post-generation colour-contrast checker rejects WCAG-AA-failing combinations and falls back to a safe palette.	—
5	Password-reset tokens exposed in HTTP response when `DEV_MODE=true`	High (config)	**CLOSED** (2026-04-24)	Call `POST /api/auth/forgot-password` with `DEV_MODE=true`. Token appears in body.	Default for `DEV_MODE` is `false`. Prod task def must keep it `false`.
6	IDOR across deploy and section endpoints	High (security)	**CLOSED** (2026-04-24)	Get a JWT for user A. `POST /api/deploy` with user B's `session_id`.	All `/api/sections/*` and the main `/api/deploy` now verify `session.user_id == auth.user_id`.
7	SQL injection surface via tenant schema name	High (security)	**CLOSED**	A trade licence with a `"` could break out of `SET search_path TO "{schema}"`.	`database._validate_schema_name` rejects anything not `[a-zA-Z0-9_]+`.
8	Content moderation fails open on LLM error	Medium (security)	**CLOSED** (2026-04-24)	Block egress to OpenAI; submit a prompt that would normally be rejected.	`check_prompt_safety_llm` now fails **closed**. Operators can flip `MODERATION_LLM_ENABLED=false` for keyword-only.
9	Exception details leaked to client in deploy + admin routes	Medium (security)	**CLOSED** (2026-04-24)	Trigger any backend exception. Response body contained the boto3 traceback.	`deploy.py` and 20+ admin endpoints now return generic messages; tracebacks logged via `logger.exception`.
10	Weak password policy	Medium (security)	**CLOSED** (2026-04-24)	Sign up with `abc123`. Accepted.	`validate_password_strength` enforces 12 chars + upper + lower + digit + symbol. Wired into signup, reset-password, change-password.
11	Path traversal in file delete	Medium (security)	**CLOSED** (2026-04-24)	`DELETE /api/uploads/../../etc/passwd`.	`storage.delete_file` resolves path against `UPLOAD_DIR` and rejects escapes.

Open configuration / CI issues

#	Title	Severity	Fix
C1	**`SONAR_TOKEN` missing from GitHub repo secrets**	Low	SonarCloud Analysis job in `ci.yml` is red on every push. Add the token at Settings → Secrets → Actions, or remove the SonarCloud step.

#	Title	Severity	Fix
C2	**`deploy-prod.yml` `wait services-stable` occasionally times out on a successful deploy**	Low	The `aws ecs wait` polls 40 × 15 s = 10 min. Slow rollouts exceed it. Treat as green if `/api/health` returns 200 within 5 min of push. Permanent fix: raise the wait or move to a follow-up step polling `/api/health`.
C3	**Legacy `production` and `staging` branches** unwired	Cosmetic	Safe to delete: `git push origin --delete production staging`.
C4	**me-central-1 leftover ElastiCache cluster stuck in `creating`**	Low	`meydan-prod-redis` in me-central-1 (UAE) has been in `creating` since 2026-04-23. AWS refuses delete in this state. Open AWS Support ticket — text prepared during migration session.
C5	**me-central-1 leftover S3 bucket** `meydan-prod-deployed-sites`	Cosmetic	192 objects, 48 MB, charges pennies. Delete after Singapore is confirmed serving live for 30 days.

Unfinished items — current state

Item	State	Notes
Prompt Manager — NOT WIRED, NOT TESTED ⚠️	UI renders. DB tables (`prompt_versions`) accept writes. Admin endpoints `GET/POST /api/admin/prompts/*` work and update the DB. **The Worker / generation pipeline does not read from the DB at all** — it uses hardcoded prompts in `backend/app/utils/prompt_pipeline.py`. Publishing a new version through the UI changes nothing about generation. End-to-end tests for the wiring do not exist.	Until wired, the only way to change a prompt is to edit `prompt_pipeline.py`, commit, and push to `main`. Wiring work: (1) make `prompt_pipeline.py` read active version from `prompt_versions` table with 60s in-process cache and fall-back to in-file defaults; (2) add a "preview with draft prompt" admin endpoint; (3) add unit test covering publish + rollback affecting generation output. ~1 day total. **This is the top unfinished item in the handover.**
Token system rationalisation	Both "generation limit" and "token budget" enforcement paths exist.	Client to confirm whether tokens stay or just per-month generation cap. If tokens go: drop `token_transactions` charging logic; keep `users.max_generations_per_month`.
File-upload generation (PDF / text / images)	Backend upload endpoints exist (`/api/uploads/*`) but the generator does not ingest uploads into the Stage 0 brief.	Needs PDF / DOCX text extraction + image-placeholder mapping.
AI-Agent FAQ on landing page	Design decided; not built.	Replacement for the static FAQ accordion.
E-commerce module	Architectural plan exists in `docs/ecommerce-feature-plan.{md,pdf}`. Not started.	~9-week MVP. Parked pending client sign-off.
Capacity plan: 500 – 1,000 concurrent generations	Sizing analysis done. Not implemented.	Critical path is **LLM provider tier upgrades** — OpenAI Tier 4+ ($250 paid, 14d wait) and Anthropic Tier 4 + Higher Rate Limits form (1–2 weeks). Without those, AWS sizing is moot — 429s cap throughput at ~50 concurrent.
Refresh tokens	Not implemented.	Users are forced to re-login every `JWT_EXPIRE_MINUTES`.

Technical debt — recommended for revisiting

- **`backend/app/routes/generator.py`** is >4,000 lines and owns too many concerns (session, streaming, SSE, moderation, file ingest). Split into `generator_stream.py`, `generator_session.py`, `generator_uploads.py`.
- **`backend/app/routes/admin_old.py`** is the previous-gen admin router; current admin lives in `backend/app/routes/admin/`. `admin_old.py` is still mounted for back-compat. Plan a deprecation sweep.
- **Tenant schema DDL** is applied on signup by raw SQL. Move through Alembic templated migrations so a schema-repair tool can diff and apply.
- **Dev-side secrets injection** still uses plaintext env vars in the ECS task definition — Singapore prod is on Secrets Manager. Migrate dev to the same pattern.
- **`backend/debug_output/*`** (generated site artefacts) should be moved to S3 or dropped from the repo.
- **Tailwind post-processing** (`premium_postcss.py`) is bespoke. Tailwind v4 / v5 will need rework.
- **Stale Route 53 zones** exist in dev account from earlier setup. The prod account hosted zone is authoritative; clean up dev account zones to avoid confusion.

Support contacts — post-handover support period

The valid contact window is the agreed support period in the master services agreement (typically 30 days from delivery). Beyond that, contact the Meydan AI lead.

Name	Role	Email	Phone	Response window
Ashutosh	Technical lead (vendor) — backend, generation pipeline, deploy	ashuken266@gmail.com	*(in workbook)*	Same-business-day, weekdays 09:00–18:00 GST
(Frontend engineer)	Frontend, editor, admin UI	*(tbd)*	*(tbd)*	Same-business-day
(DevOps / AWS contact)	Infra, incidents, CloudWatch, AWS billing	*(tbd)*	*(tbd)*	1 h pager for production outages (P1)
(Meydan AI lead)	Client-side owner — escalation, scope, next phase	*(tbd)*	*(tbd)*	1 business day

> The workbook's Contacts sheet is the authoritative list — if this document disagrees, the workbook wins.

How to escalate during an outage

1. Check CloudWatch dashboard `meydan-production` (AWS console, `ap-southeast-1`) — red panels show what is broken.
2. Check provider status pages: `status.anthropic.com`, `status.openai.com`, `status.aws.amazon.com`, `status.resend.com`.
3. If an LLM circuit breaker is stuck open: `POST /api/admin/circuit/{openai|anthropic}/reset`.
4. If the site is entirely down: check ALB target health in the AWS console; if targets are unhealthy, `aws ecs update-service --force-new-deployment` for the affected service.
5. If DNS is the problem: the authoritative zone is **CloudFront**, not Route 53.
6. For data incidents, follow the DR playbook in `infrastructure/aws/OPERATIONS-GUIDE.md`.

A correlation ID from the failed request (returned as `X-Correlation-ID` on every response) is the fastest way to trace a user-reported issue through CloudWatch Logs.

Disaster Recovery Runbook — Singapore Production

Supplementary to the 10-document handover. Covers what is protected, what is not, and how to restore.

What is protected today

Component	Protection	Where it lives
RDS PostgreSQL	Multi-AZ (auto-failover within region) + 14-day automated backups + KMS encryption + deletion protection	`ap-southeast-1`, account `290172088819`
ElastiCache Redis	7-day automated snapshot retention	`ap-southeast-1`
Customer-deployed sites (S3)	Versioning **enabled** (any object overwrite/delete is recoverable for 90 days), 99.999999999% intra-region durability	`s3://meydan-prod-sites-sg`
ECR images	Lifecycle keeps last 20 images per repo; untagged auto-expire at 7 days	`290172088819.dkr.ecr.ap-southeast-1.amazonaws.com/{meydan-api,meydan-frontend}`
Secrets	Secrets Manager — encrypted at rest, multi-AZ replicated	`meydan/prod/*`
Code + IaC	Git on GitHub `Meydanfz23/meydan-website-generator`	branches `dev`, `main`
Configuration	ECS task definitions are revision-controlled; `aws ecs describe-task-definition --task-definition meydan-prod-{api,worker,frontend}:N` returns any historical revision	ECS

What is NOT protected (accepted risk at handover)

- **Region-wide outage of `ap-southeast-1`** — no cross-region snapshot copies. The blast radius is the entire prod stack. Mitigations declined for cost (~$10–25/mo). If risk profile changes, set up an AWS Backup plan with copy to `ap-southeast-2` (Sydney) — instructions below under "Adding cross-region copies later".
- **Drone-strike-class infrastructure damage** — same as above. The reason `ap-southeast-1` was chosen over `me-central-1` is geopolitical stability, not engineering invulnerability.
- **Account-wide compromise** — if the prod IAM user is hijacked, an attacker with admin can delete data and snapshots in one region. Defence: enable MFA on the root + IAM user, and add CloudTrail-driven alarms for `Delete*` API calls.

Backup inventory commands

Run these to verify daily that backups are flowing:

```
# Latest RDS automated snapshot
aws rds describe-db-snapshots --db-instance-identifier meydan-prod-db \
  --snapshot-type automated --region ap-southeast-1 --profile prod \
  --query 'DBSnapshots | sort_by(@, &SnapshotCreateTime) | [-1].{id:DBSnapshotIdentifier,created:SnapshotCreateTime}'

# RDS PITR window (the earliest restorable point)
aws rds describe-db-instances --db-instance-identifier meydan-prod-db \
  --region ap-southeast-1 --profile prod \
  --query 'DBInstances[0].{earliest:EarliestRestorableTime,latest:LatestRestorableTime}'

# Latest ElastiCache snapshot
aws elasticache describe-snapshots --cache-cluster-id meydan-prod-redis \
  --region ap-southeast-1 --profile prod \
  --query 'Snapshots | sort_by(@, &NodeSnapshots[0].SnapshotCreateTime) | [-1].{name:SnapshotName,created:NodeSnapshots[0].SnapshotC

# S3 object versions (sample)
aws s3api list-object-versions --bucket meydan-prod-sites-sg --max-items 5 --profile prod \
  --query 'Versions[].{key:Key,id:VersionId,modified:LastModified,latest:IsLatest}'
```

Recovery procedures

Scenario 1 — "A user reports their deployed site has been overwritten with a bad version"

```
# 1. List versions for that customer's index.html
aws s3api list-object-versions --bucket meydan-prod-sites-sg --prefix <slug>/index.html --profile prod

# 2. Restore by copying the previous version on top of the current key
aws s3api copy-object --bucket meydan-prod-sites-sg \
  --copy-source "meydan-prod-sites-sg/<slug>/index.html?versionId=<previous-version-id>" \
  --key "<slug>/index.html" --profile prod

# 3. Invalidate the user's CloudFront distribution if they're on a custom domain
aws cloudfront create-invalidation --distribution-id <id> --paths "/index.html" --profile prod
```

Recovery time: **under 5 minutes per site**. Recovery point: **last 90 days of versions**.

Scenario 2 — "We need to roll back the database to before a bad migration ran"

Two paths.

Path A — Point-in-time recovery (fast, preferred): restores the DB to any second within the 14-day backup window.

```
# 1. Restore to a NEW instance (cannot overwrite running prod)
aws rds restore-db-instance-to-point-in-time \
  --source-db-instance-identifier meydan-prod-db \
  --target-db-instance-identifier meydan-prod-db-restore \
  --restore-time 2026-04-27T18:30:00Z \
  --multi-az --db-subnet-group-name meydan-prod-db-subnet \
  --vpc-security-group-ids <DB_SG> \
  --no-publicly-accessible \
  --region ap-southeast-1 --profile prod

# 2. Wait for status=available (~10 min), then verify the data
aws rds wait db-instance-available --db-instance-identifier meydan-prod-db-restore \
  --region ap-southeast-1 --profile prod

# 3. Cut over: scale ECS API + Worker to 0, repoint Secrets Manager `meydan/prod/database-url`
#    at the new endpoint, force-redeploy. Keep the old instance for 7 days as belt-and-braces.
```

Recovery time: **~15 minutes** (10 min restore + 5 min cut-over). Recovery point: **any second in the last 14 days**.

Path B — Restore from a specific automated snapshot: same flow, replace `restore-db-instance-to-point-in-time` with `restore-db-instance-from-db-snapshot --db-snapshot-identifier <id>`.

Scenario 3 — "An attacker / intern dropped the production database"

Deletion protection is on, so `aws rds delete-db-instance` will refuse without `--skip-final-snapshot --no-deletion-protection` — both intentional friction. If it still happens:

1. Check the **automated snapshots** — they survive instance deletion for the retention period.
2. Restore the latest automated snapshot to a new instance per Scenario 2 Path B.
3. Repoint Secrets Manager and redeploy.

Recovery time: **~15 minutes**. Recovery point: **last automated snapshot (≤24 h old)**.

Scenario 4 — "The Singapore region is down"

This is the gap we accepted. Without cross-region snapshots, Singapore-down means the app is down until AWS recovers. Customer-deployed sites continue serving (they are S3 + intra-region replicated; an S3 region outage is rare but possible).

The plan if `ap-southeast-1` becomes a multi-day issue:

1. Stand up replacement infrastructure in `ap-southeast-2` (Sydney) using `infrastructure/aws/` templates — VPC, ECR, ALB, etc. ~1 hour.
2. Wait for `ap-southeast-1` access to return enough to take a manual `pg_dump` and `s3 sync`. If Singapore is genuinely lost, accept data loss back to last cross-region replication (which is none — we declined that protection).
3. Repoint Route 53 / CloudFront origin to the Sydney ALB.

This scenario is the reason cross-region copies are recommended even for small accounts. We've accepted the risk; if Singapore becomes problematic we revisit.

Adding cross-region copies later (when ready)

```
# Create AWS Backup vault in destination region first
aws backup create-backup-vault --backup-vault-name meydan-prod-dr-sydney \
  --region ap-southeast-2 --profile prod

# Then in source region, define a plan that copies daily
aws backup create-backup-plan --region ap-southeast-1 --profile prod \
  --backup-plan '{
    "BackupPlanName": "meydan-prod-daily-with-cross-region-copy",
    "Rules": [{
      "RuleName": "DailyBackup",
      "TargetBackupVaultName": "Default",
      "ScheduleExpression": "cron(0 19 ? * * *)",
      "StartWindowMinutes": 60,
      "CompletionWindowMinutes": 180,
      "Lifecycle": { "DeleteAfterDays": 30 },
      "CopyActions": [{
        "DestinationBackupVaultArn": "arn:aws:backup:ap-southeast-2:290172088819:backup-vault:meydan-prod-dr-sydney",
        "Lifecycle": { "DeleteAfterDays": 30 }
      }]
    }]
  }'

# Then assign resources (RDS + ElastiCache) to the plan via aws backup put-backup-selection.
```

Cost at our scale: ~$10–25 / month.

Restore drill (recommended quarterly)

Once a quarter, run Scenario 2 Path A end-to-end into a non-prod restore instance. Drop it after verifying. This proves the backups are real and tests the procedure when the team has time, not at 3 AM mid-incident.

Sign-off

Item	Status	Verified by	Date
RDS Multi-AZ + 14-day backups + deletion protection	✓	Migration session	2026-04-26
ElastiCache 7-day snapshots	✓	Migration session	2026-04-26
S3 versioning + 90-day non-current expiry	✓	DR hardening	2026-04-27
ECR lifecycle (20 images, 7-day untagged expiry)	✓	DR hardening	2026-04-27
Cross-region snapshot copies	× (accepted, $10–25/mo declined)	—	2026-04-27
Restore drill	× (recommended quarterly)	—	—